



Deep Learning mit Python & TensorFlow

DR. CHIAO-YA CHANG
CHIAOYA.CHANG.TW@GMAIL.COM

Tag 1 – Grundlagen Python





Tag 1 10:00–10:30

Begrüßung & Einführung

- Erwartungskklärung
- Kursziele
- Ablauf
- Vorstellungsrunde

Erwartungskklärung:

Keine Vorkenntnisse in Programmierung oder Python erforderlich

Die Teilnehmenden sollten jedoch:

- **Neugier und Interesse** an Künstlicher Intelligenz und neuen Technologien mitbringen
- bereit sein, **aktiv mitzuarbeiten und selbst auszuprobieren**
- grundlegende **PC-Kenntnisse** besitzen (z. B. Umgang mit Maus, Tastatur und Dateien)
- sich darauf einstellen, dass Programmieren ein **Lernprozess mit Übung und Geduld** ist

Der Kurs richtet sich an Anfänger*innen und legt Wert auf eine **ruhige, verständliche und unterstützende Lernatmosphäre**.



Kursziele:

Einen verständlichen & praxisnahen Einstieg in Deep Learning ermöglichen.

Am Ende des Kurses sollen die Teilnehmenden:

- die **Grundlagen der Programmiersprache Python** verstehen und einfache Programme lesen und anpassen können
- ein grundlegendes **Verständnis von Künstlicher Intelligenz und Deep Learning** entwickeln
- wissen, wie neuronale Netze aufgebaut sind
- erste Erfahrungen mit **TensorFlow** sammeln
- ein **einfaches neuronales Netz selbst erstellen, trainieren und auswerten**
- einschätzen können, **wie Deep Learning in der Praxis eingesetzt** wird und wie sie ihr Wissen weiter vertiefen können



Ablauf

Tag 1 – Python & Datenanalyse			Tag 2 – Deep Learning		
10:00–10:30	Begrüßung & Einführung	Kursziele, Ablauf, Erwartungskklärung, Vorstellungsrund	10:00–10:30	Rückblick & Zielsetzung	Wiederholung des Tag1
10:30–11:30	Python-Grundlagen	Datentypen, Variablen, Schleifen, Funktionen	10:30–11:00	Grundlage einer Modelle	Parameter berechnen, Regression vs. Klassifikation Overfitting vs. Underfitting
11:30–12:30	Arbeiten mit Daten (Pandas)	Daten laden, filtern, berechnen	11:00–12:30	Verständliche Erklärung: Was ist Deep Learning und wie funktioniert es?	Neuronale Netze, TensorFlow Grundlagen
12:30–13:10	 Mittagspause	—	12:30–13:10	 Mittagspause	—
13:10–15:00	Datenvisualisierung	Matplotlib / Seaborn: Diagramme & Insights	13:10–15:00	Einführung in Deep Learning	Neuronale Netze, TensorFlow Grundlagen
15:00–15:20	 Pause	—	15:00–15:20	 Pause	—
15:20–16:40	Mini-Projekt: Explorative Analyse mit Python	Gruppenarbeit mit realem Datensatz	15:20–16:40	Praktische Übung: Einfaches Modell	Ergebnisse analysieren, Verbesserungen diskutieren
16:40–17:00	Ergebnisse & Diskussion	Präsentation, Reflexion, Q&A	16:40–17:00	Ergebnisse & Diskussion	Präsentation, Reflexion, Q&A



Vorstellungsrunde: Über mich

Dr. Chiao-Ya Chang

- **Promotion** in Agra-Ökonomie an der Universität Bonn
- Freiberufliche Data Spezialistin & Trainerin für Datenanalyse und Künstliche Intelligenz
- Supply Chain Analytics and Simulation Manager in der Crop Science Abteilung bei Bayer AG
- Commercial Data Scientist in der Pharmaceutical Abteilung bei Bayer AG
- Data Scientist bei Lidl Stiftung
- **Webseite:** www.drchiaoyachang.com

Vorstellungsrunde:

Wir lernen uns kennen

- **Name**
- **Erfahrungen mit Python und Deep Learning**
- **Motivation**
- **Erwartungen**
- **Sonstiges**





Tag 1 10:30–11:30

Python-Grundlagen

- Datentypen
- Variablen
- Schleifen
- Funktionen

Datentabellen verstehen

Grundlage der Datenanalyse

1 **Daten sind überall** -- aber oft schwer zu verstehen

2 **Ohne Struktur** -- keine Analyse möglich

3 **Tabellen sind der Einstieg** in jede Datenanalyse



Daten sind überall

In jedem Shop-Kauf, jeder App-Nutzung, jedem Klick steckt ein Datensatz.

Grundstruktur einer Datentabelle

Spalten (Columns) = Eigenschaften

- **order_id** eindeutige Nummer
- **customer_id** wer hat bestellt
- **product_id** was wurde bestellt
- **quantity** wie viel

Zeilen (Rows) = einzelne Einträge

order_id	customer_id	product_id	quantity
1001	14	120	4
1002	20	110	3
1003	4	105	4
1004	12	101	4

↑ jede Zeile = eine Bestellung (ein Ereignis)

Aufgabe: Python Installation

- Link: <https://www.python.org/>
- Jupyter Notebook file: file_name.**ipynb**
- Python file: file_name.**py**
- **Hausaufgaben:** Python installation mit privaten PC/Laptop
- **Kursmaterial (GitHub)**





Tag 1 11:30–12:30

Arbeiten mit Daten (Pandas)

- Daten laden, filtern, berechnen

Was ist Pandas?

Warum Pandas verwenden?

Pandas ist eine Python-Bibliothek, die zur Arbeit mit **Datensätzen** verwendet wird.

Pandas bezieht sich sowohl auf „**Panel Data**“ als auch auf „**Python Data Analysis**“.

Funktionen zum:

- Analysieren, bereinigen und erkunden von Daten
- Verändern und Bearbeiten von Daten

Pandas ermöglicht es uns:

- große Datenmengen zu analysieren
- Schlussfolgerungen auf Basis statistischer Methoden zu ziehen



12:30–13:10 ☕ Mittagspause



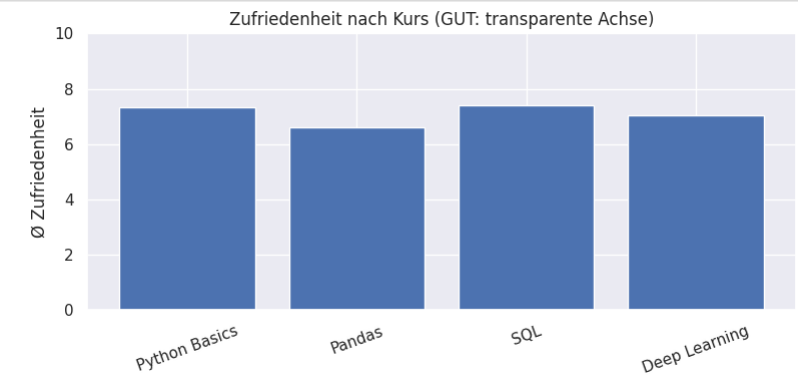
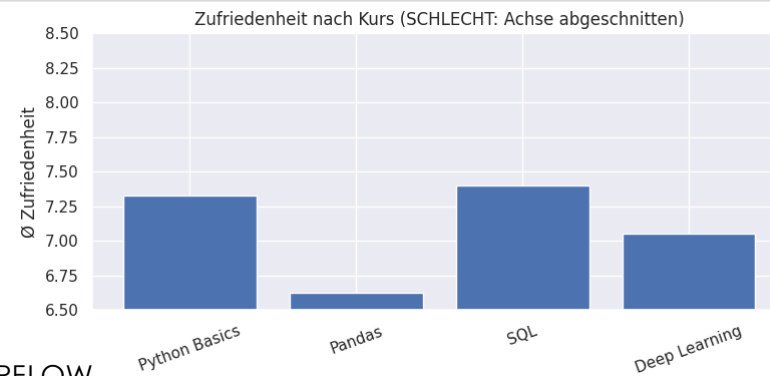
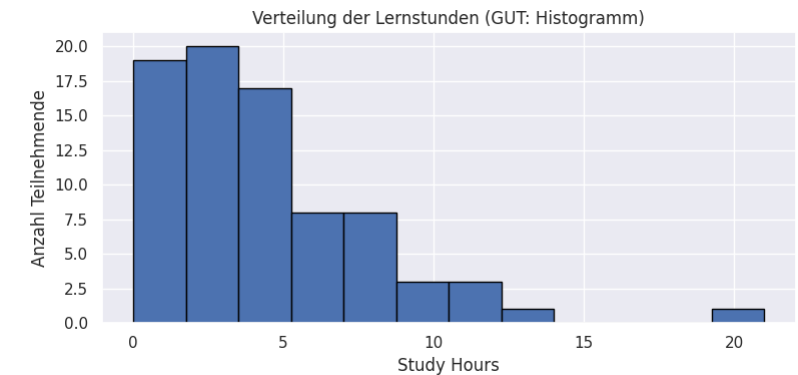
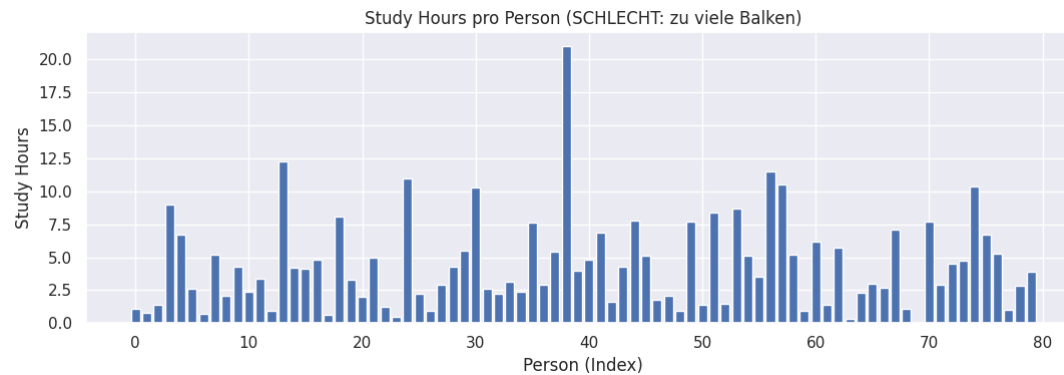
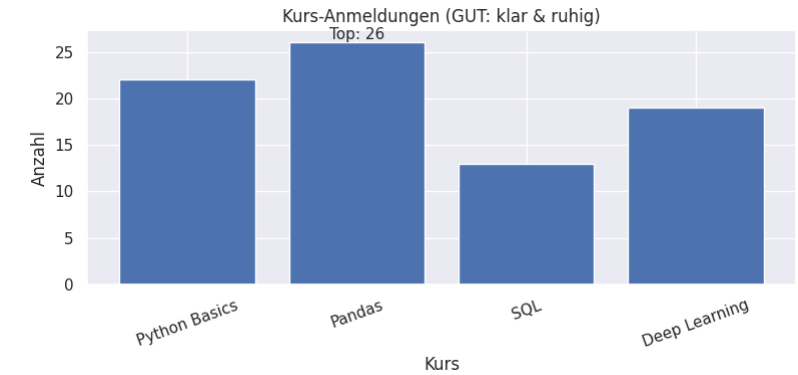
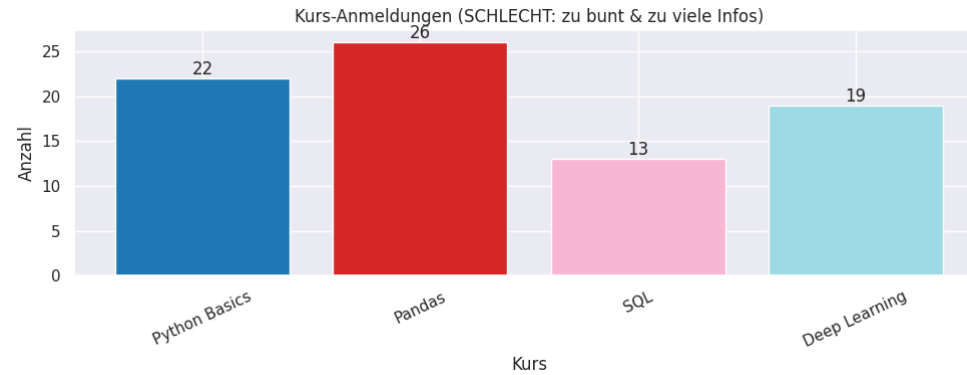


Tag 1 13:10–15:00

Datenvisualisierung

- Matplotlib / Seaborn
- Diagramme & Insights

Gute vs. schlechte Visualisierung



15:00–15:20 🍴 Pause





Tag 1 15:20–16:40

Mini-Projekt: Explorative Analyse mit Python

- Gruppenarbeit mit realem Datensatz



Tag 1 16:40–17:00

Ergebnisse & Diskussion

- Präsentation
- Reflexion
- Übergang zum Deep Learning
- Q&A

Übergang zum Deep Learning

mini_bild: 4×4 Pixel (0 = schwarz, 255 = weiß)

0	50	100	150
200	255	30	60
90	120	180	210
10	40	70	240

Tag 2 – Gemeinsames Erstellen eines Neuronalen Netzes





Tag 2 10:00–10:30

Rückblick, Übung und Zielsetzung

- Wiederholung
- Einführung in den zweiten Tag



Tag 2 10:30–11:00

Grundlage einer Modelle

- Parameter der Linie berechnen
- Regression und Klassifikation
- Overfitting vs. Underfitting

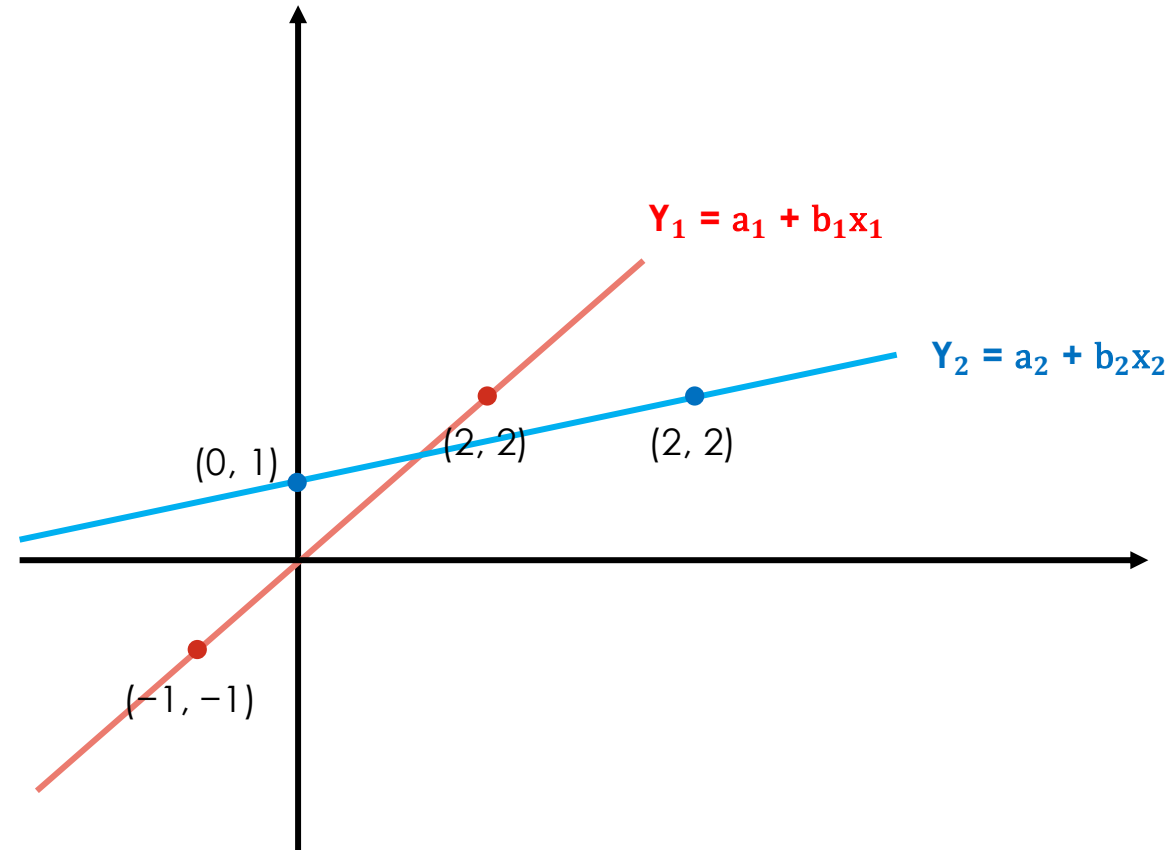
Parameter einer Linie berechnen

Grundform: $y = a + bx$

- Steigung: $b = (y_2 - y_1) / (x_2 - x_1)$
- Achsenabschnitt: $a = y_1 - b \cdot x_1$

Aufgabe: berechne a und b:

- a) Punkte (2, 2) und (-1, -1)
 - $a_1?$ $b_1?$
- b) Punkte (4, 2) und (0, 1)
 - $a_2?$ $b_2?$



Was ist ein Modell – und wie gut ist es?

Was ist ein Modell? (Black Box?)

- Eine Rechenvorschrift: Input → Berechnung → Output
- Hat Parameter, die beim Training angepasst werden

Beispiele:

- $y = a + bx$ – eine Gerade
- Neuronales Netz mit Gewicht w und Bias b

Wie gut ist es? (Fehler oder Treffer?)

- Regression: Fehler = Abstand zur echten Zahl
- Klassifikation: Accuracy = Anteil richtiger Antworten

Beispiele:

- Kleiner Fehler = gute Vorhersage von Zahlen
- Hohe Accuracy = gute Vorhersage von Kategorien
- Fachbegriff im Code: Loss (z. B. `sparse_categorical_crossentropy`)

Wie gut ist ein Modell?

1. Wettervorhersage: 3 °C vorhergesagt, real waren es 5 °C.
2. Spam-Filter: 98 von 100 E-Mails richtig eingeordnet.
3. Hauspreis-Modell: 340.000 € vorhergesagt, echter Preis 350.000 €.
4. Bildklassifikation: „Hund“ erkannt – es war wirklich ein Hund.
5. Notendurchschnitt: 2,3 vorhergesagt, echter Schnitt 2,1.
6. Krankheitserkennung: 47 von 50 Patienten richtig diagnostiziert.

Klassifikation vs. Regression

Klassifikation (Dropdown-Liste?)

- Antwort = eine Kategorie
- Feste, begrenzte Auswahl

Beispiele:

- Spam oder kein Spam
- Katze oder Hund
- T-Shirt oder Hose

Regression (Taschenrechner)

- Antwort = eine Zahl
- Unendlich viele mögliche Werte

Beispiele:

- Temperatur (23,5 °C)
- Hauspreis (350.000 €)
- Zufriedenheit (87 %)

Klassifikation vs. Regression

1. Ist diese E-Mail Spam?
2. Wie viel Grad wird es morgen haben?
3. Ist auf diesem Foto eine Katze oder ein Hund?
4. Wie viel ist dieses Haus ungefähr wert?
5. Hat der Patient Diabetes?
6. Wie hoch ist der Blutdruckwert dieses Patienten?

Overfitting vs. Underfitting

Overfitting (Auswendig gelernt?)

- Trainings-Accuracy hoch, Test-Accuracy schlecht
- Modell hat Trainingsdaten auswendig gelernt

Beispiele:

- Prüfungsfragen auswendig gelernt – neue Fragen scheitern
- Kurve geht durch jeden einzelnen Datenpunkt
- Zu komplexes Modell für wenig Daten

Underfitting (Zu einfach gedacht?)

- Trainings- UND Test-Accuracy beide schlecht
- Modell hat das Muster gar nicht richtig gelernt

Beispiele:

- Gerade Linie durch stark gekrümmte Daten gelegt (siehe Folie 23!)
- Zu wenige Trainings-Epochen
- Zu einfaches Modell für ein komplexes Problem

Overfitting vs. Underfitting

1. Trainings-Genauigkeit 99 %, Test-Genauigkeit nur 60 %.
2. Trainings- und Test-Genauigkeit sind beide nur 55 %.
3. Eine gerade Linie wird durch stark gebogene Daten gelegt.
4. Ein Modell mit tausenden Parametern wird mit nur 20 Datenpunkten trainiert.
5. Das Modell erkennt Trainingsbilder perfekt, liegt bei neuen Bildern aber fast immer falsch.
6. Das Modell erkennt selbst auf den Trainingsbildern nur die Hälfte richtig.

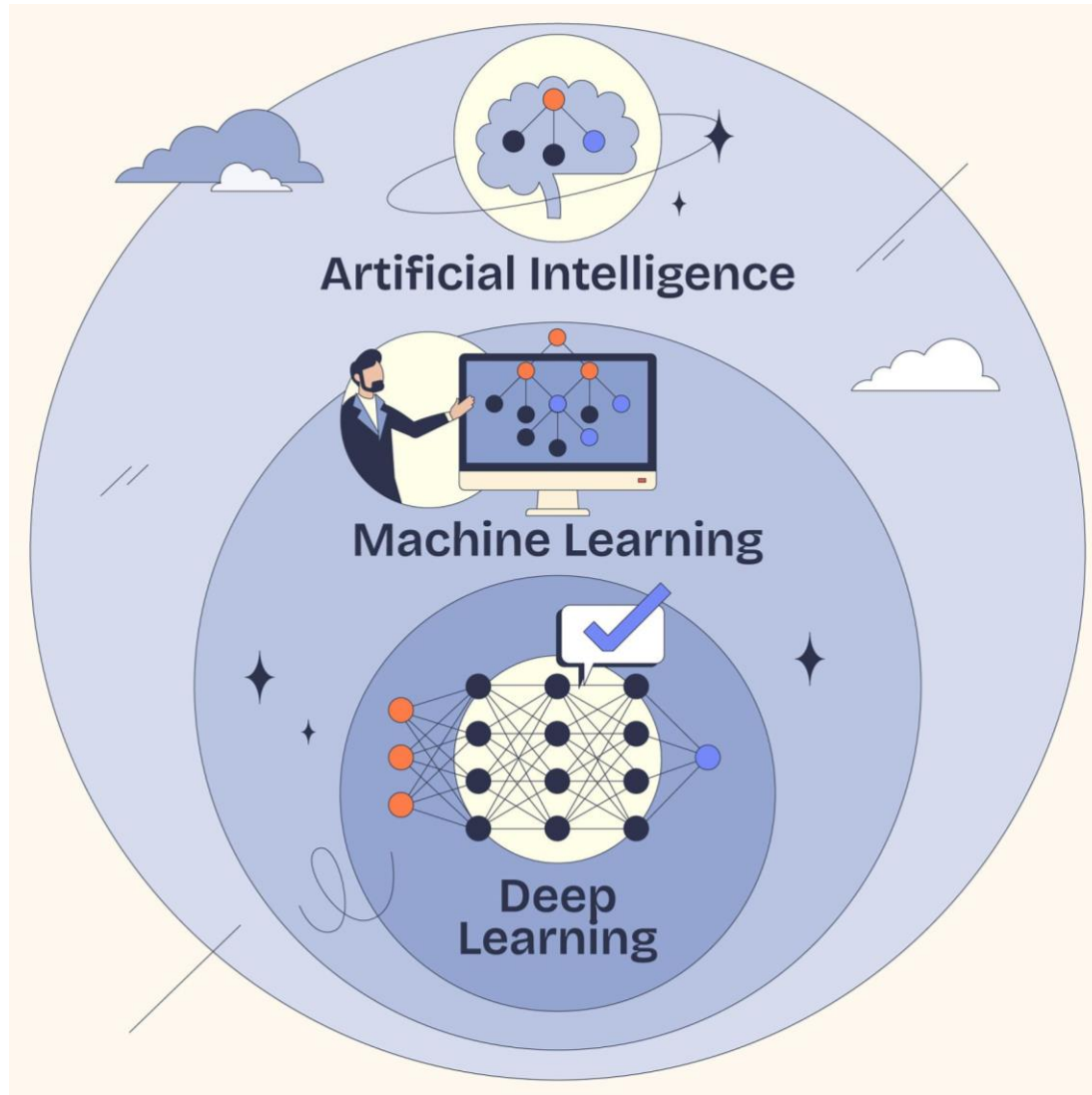


Tag 2 11:00–12:30

Verständliche Erklärung: Was ist Deep Learning und wie funktioniert es?

- Neuronale Netze
- TensorFlow Grundlagen

Künstliche Intelligenz (KI)

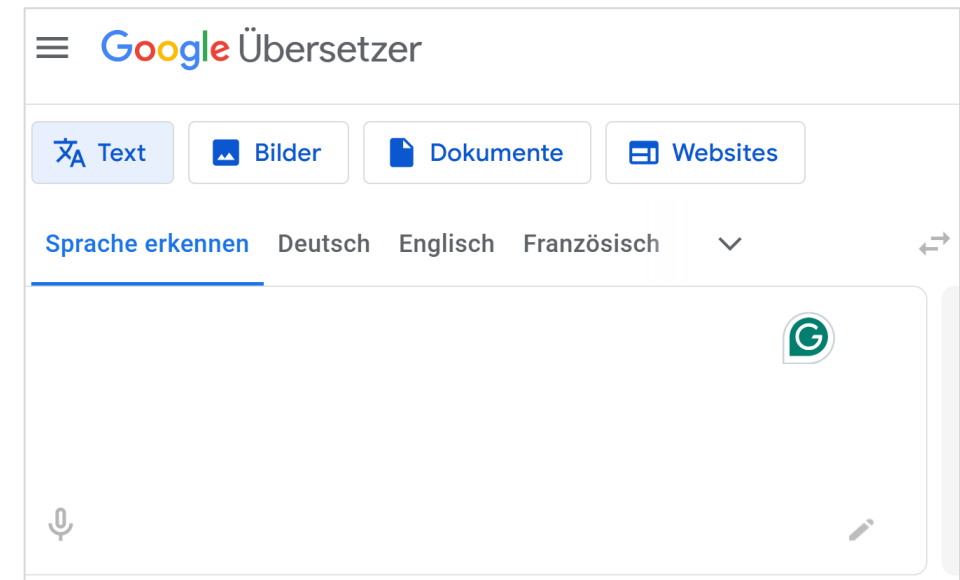
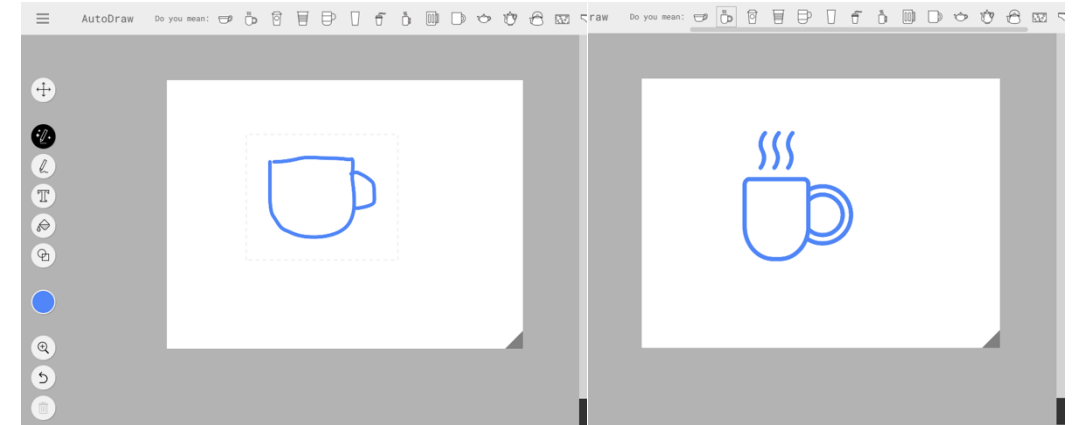


Deep-Learning-Anwendungen



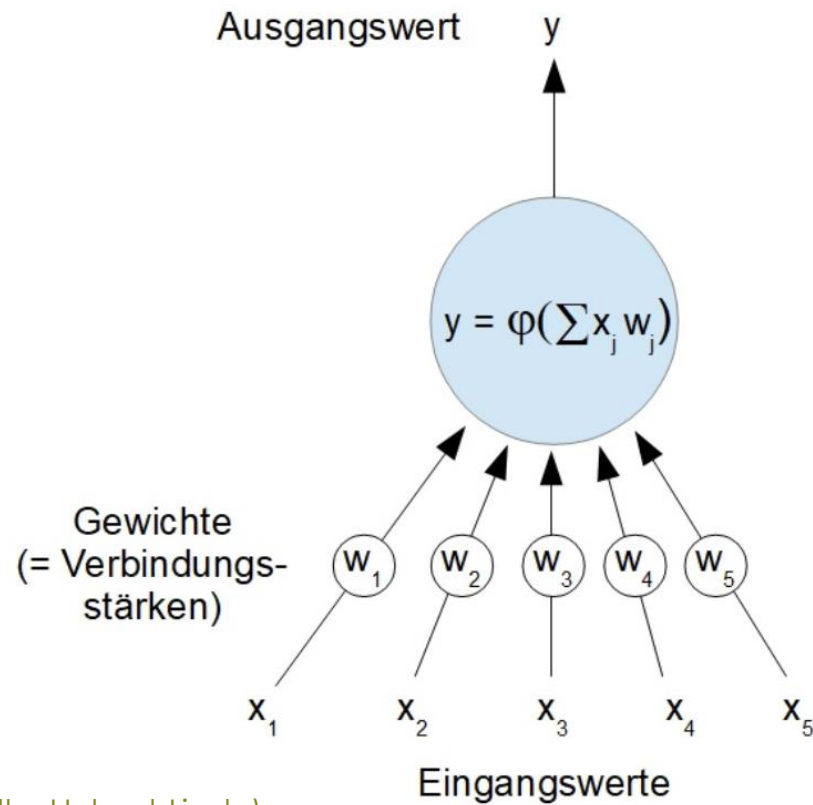
Virtual Assistant

Chatbots



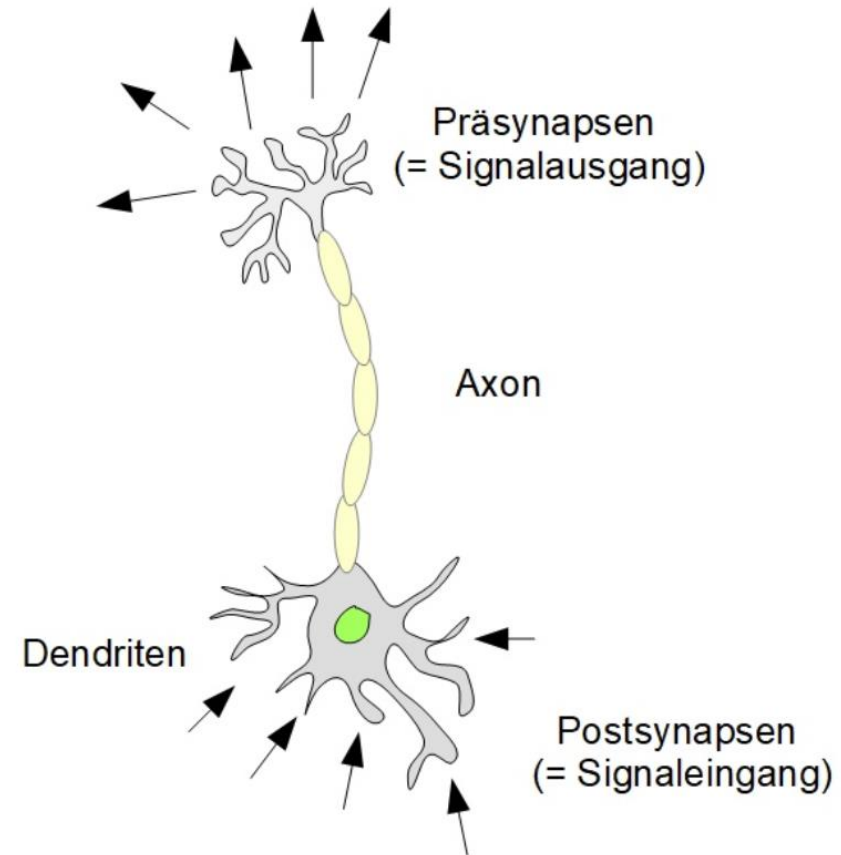
Neuron

Künstliches Neuron



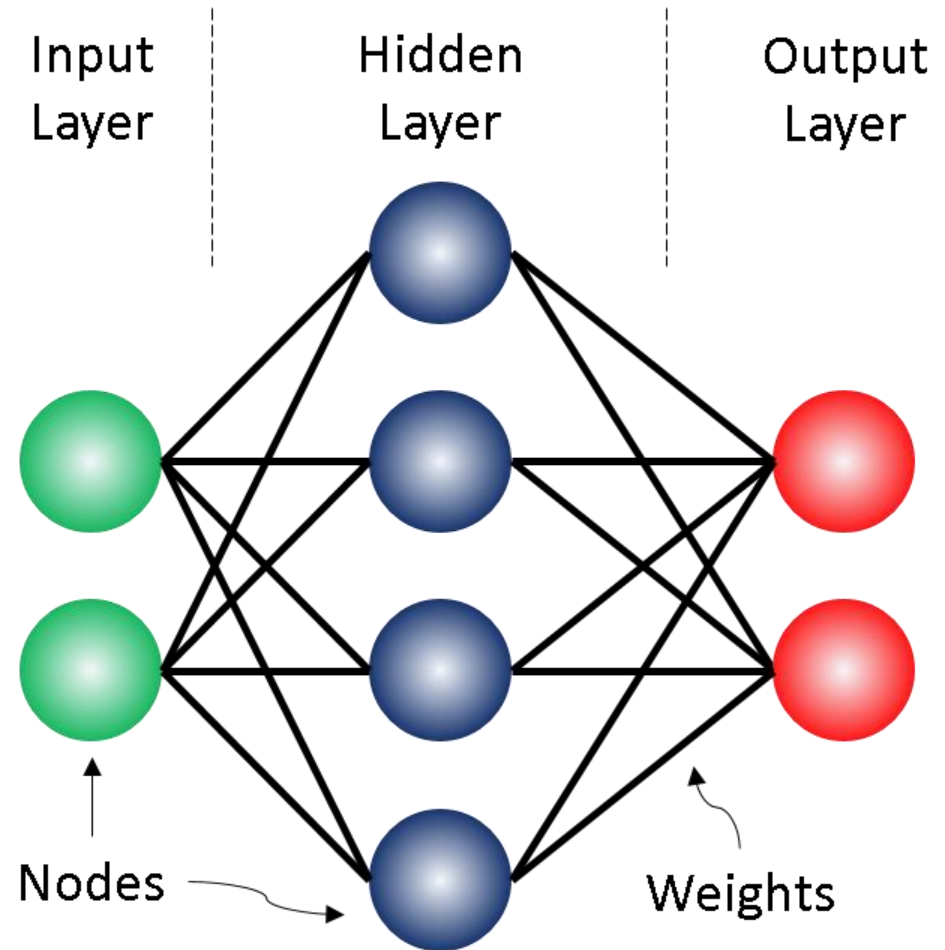
(Quelle: Helmut Linde)

Biologisches Neuron



Vom einzelnen Neuron zum neuronalen Netz

- Was ist ein neuronales Netz?
- Schichten:
 - Input Layer
 - Hidden Layer
 - Output Layer
- Warum „Deep“ = viele Schichten
- Intuition: Schichtweise Merkmalsextraktion



(Bildquelle)

TLU Perzeptron

1 Schicht

harte Schwelle (0/1)

nur linear trennbar

gut für Logik

Soll ein Alarm
ausgelöst
werden?

(TLU) Perzeptron

Output: $o = f(z)$

Step function: $f(x)$

Weighted sum: $z = \sum_{k=1}^n \mathbf{x}_k * w_k$

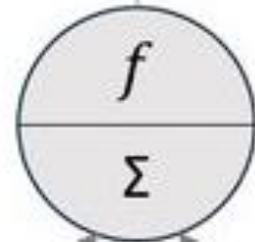
Schwellenfunktion (Threshold / Step Function)

Das ist das Herz der TLU:

$$y = \begin{cases} 1 & \text{wenn } z \geq 0 \\ 0 & \text{wenn } z < 0 \end{cases}$$

→ Nur Ja / Nein, keine Wahrscheinlichkeit.

TLU



x_1

x_2

x_3

Jede Eingabe wird **gewichtet**:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

w_i :Wichtigkeit der Eingabe

b : Bias = Verschiebung der Entscheidungsschwelle

Weights: $w_1 .. w_n$

Inputs: $x_1 .. x_n$

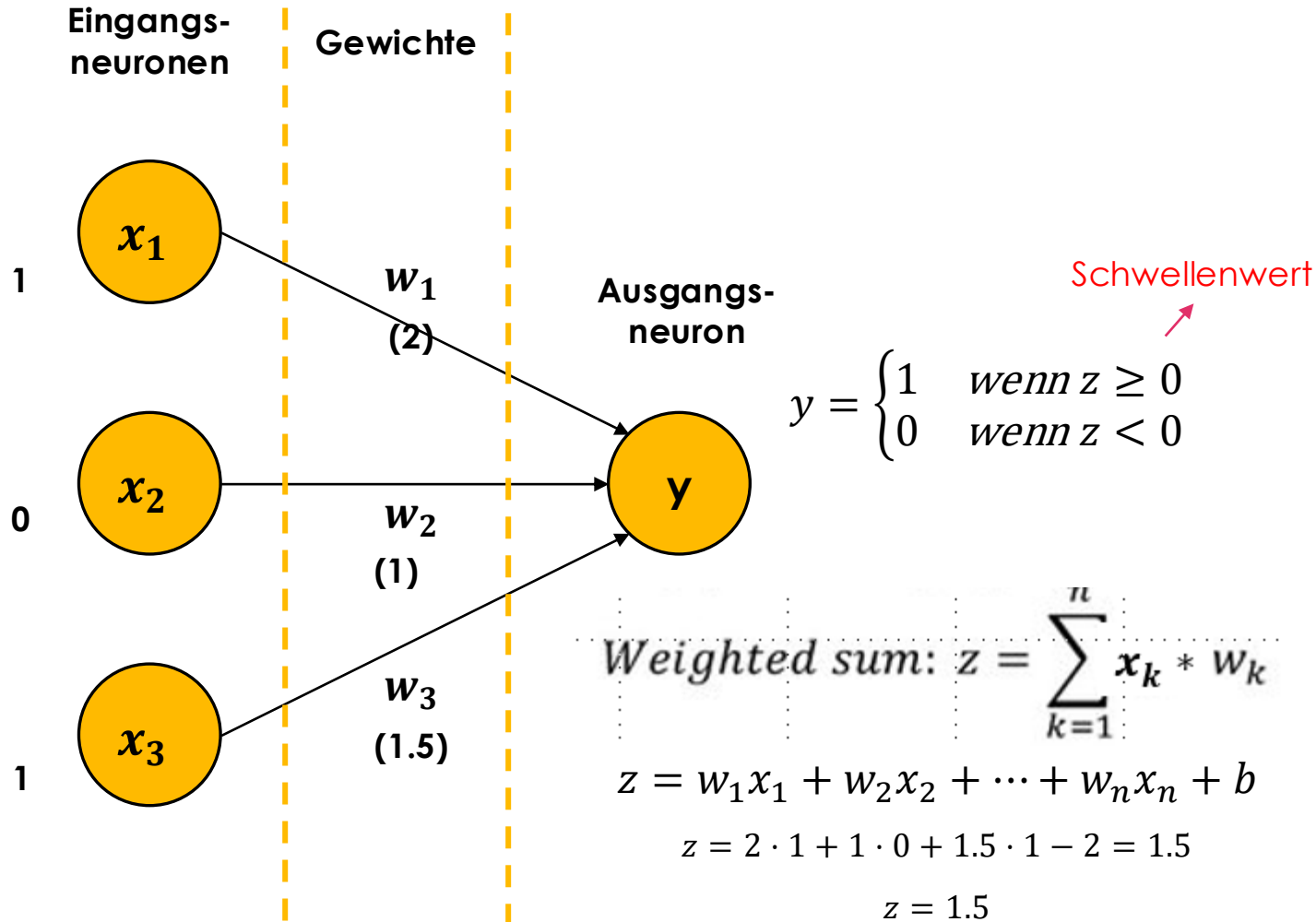
Beispiele:

x_1 :Bewegung erkannt? (0/1)

x_2 :Fenster geschlossen? (0/1)

x_3 :Nacht? (0/1)

(TLU) Perzeptron - Entscheidung



Beispiel

Soll ein Alarm ausgelöst werden?

Eingaben

$x_1 = 1 \rightarrow$ Bewegung erkannt
 $x_2 = 0 \rightarrow$ Fenster geschlossen
 $x_3 = 1 \rightarrow$ Nacht

Gewichte

$w_1 = 2$ (Bewegung ist sehr wichtig)
 $w_2 = 1$
 $w_3 = 1.5$
Bias: $b = -2$

Berechnung

$z = 2 \cdot 1 + 1 \cdot 0 + 1.5 \cdot 1 - 2 = 1.5$

$\rightarrow z \geq 0 \Rightarrow y = 1$

 Alarm aus

Frage:

Wenn der Schwellenwert = 1, was ist y?

(TLU) Perzeptron – Übungen & Diskussion

 **Situation:** Stell dir vor, du möchtest entscheiden, ob du heute ins Fitnessstudio gehst oder zu Hause bleibst?

1. Inputs (Faktoren)

Das Modell bekommt Informationen: (0 oder 1)

- x_1 : Habe ich genug Energie?
- x_2 : Habe ich Zeit?
- x_3 : Wie stark ist meine Motivation?

2. Gewichtung (Wichtigkeit)

Nicht jeder Faktor ist gleich wichtig.

- Energie → sehr wichtig (Gewicht 5)
- Zeit → mittel wichtig (Gewicht 3)
- Motivation → wichtig (Gewicht 4)

3. Schwellenwert (Threshold)

Du sagst dir:

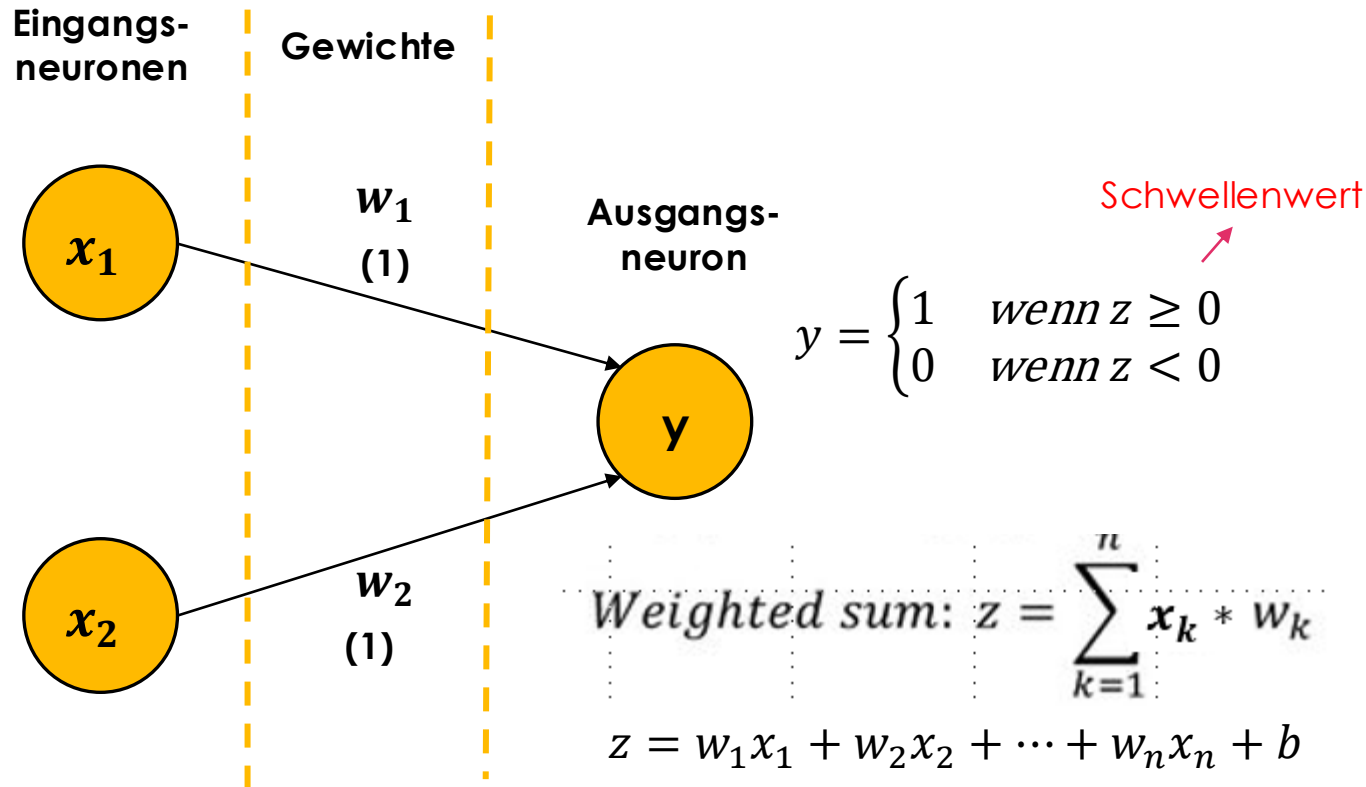
„Wenn meine Gesamtmotivation über 8 liegt, gehe ich.“

4. Entscheidung

Modell rechnet: **Faktoren × Gewichte → Summe**

Wenn **Summe ≥ Schwelle** → JA; **Sonst** → NEIN

(TLU) Perzeptron – Logik AND, OR, XOR



x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

1) AND ($b=-1.5$)

Nur wenn **beide** Eingänge aktiv sind, wird Output = 1 gefeuert

→ Große Schwellenwert

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

2) OR ($b=-0.5$)

Wenn **mindestens einer** der Eingänge aktiv ist, wird Output = 1 gefeuert

→ Kleine Schwellenwert

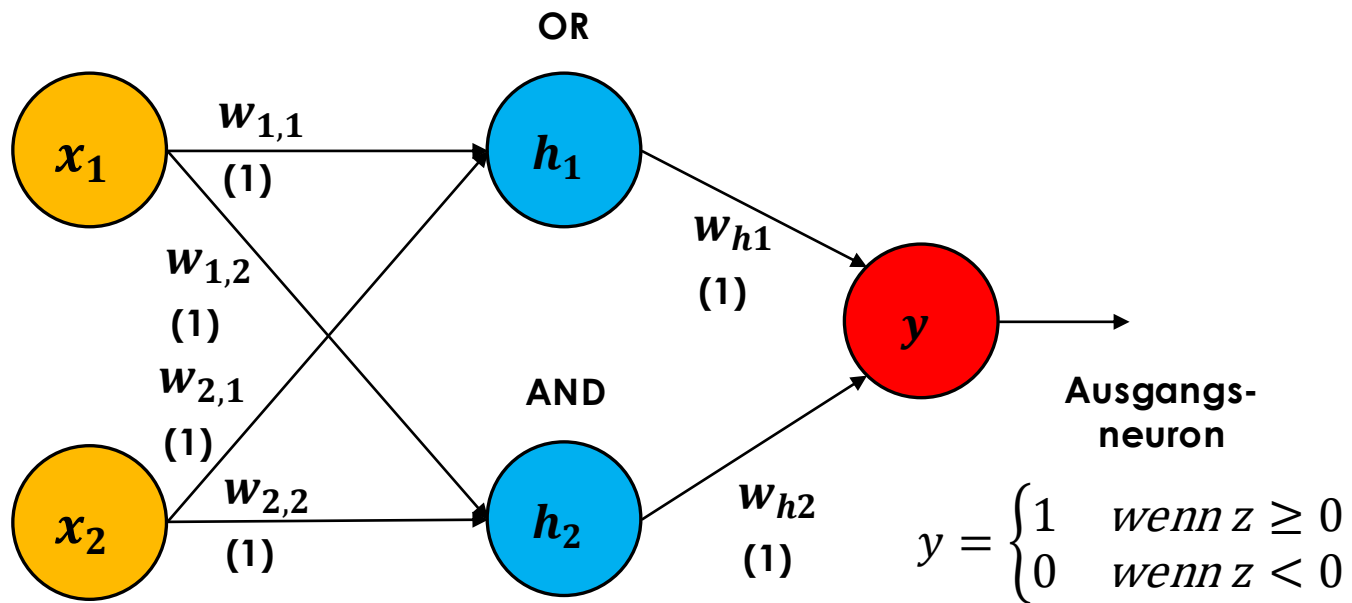
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

3) XOR

Wenn die beiden Eingaben unterschiedlich sind, wird Output = 1 gefeuert

Warum brauchen wir ein Hiddenschicht?

Eingangs-
neuronen



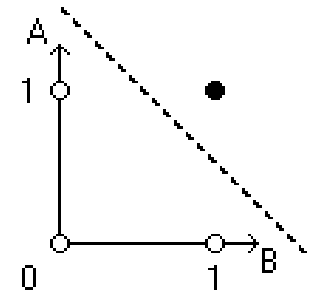
DEEP LEARNING MIT PYTHON UND TENSORFLOW

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

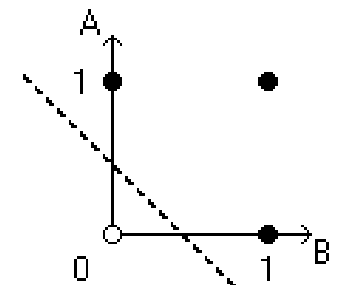
x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

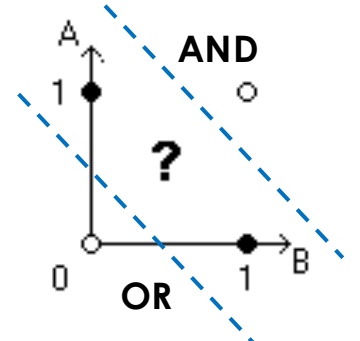
1) AND ($b=-1.5$)



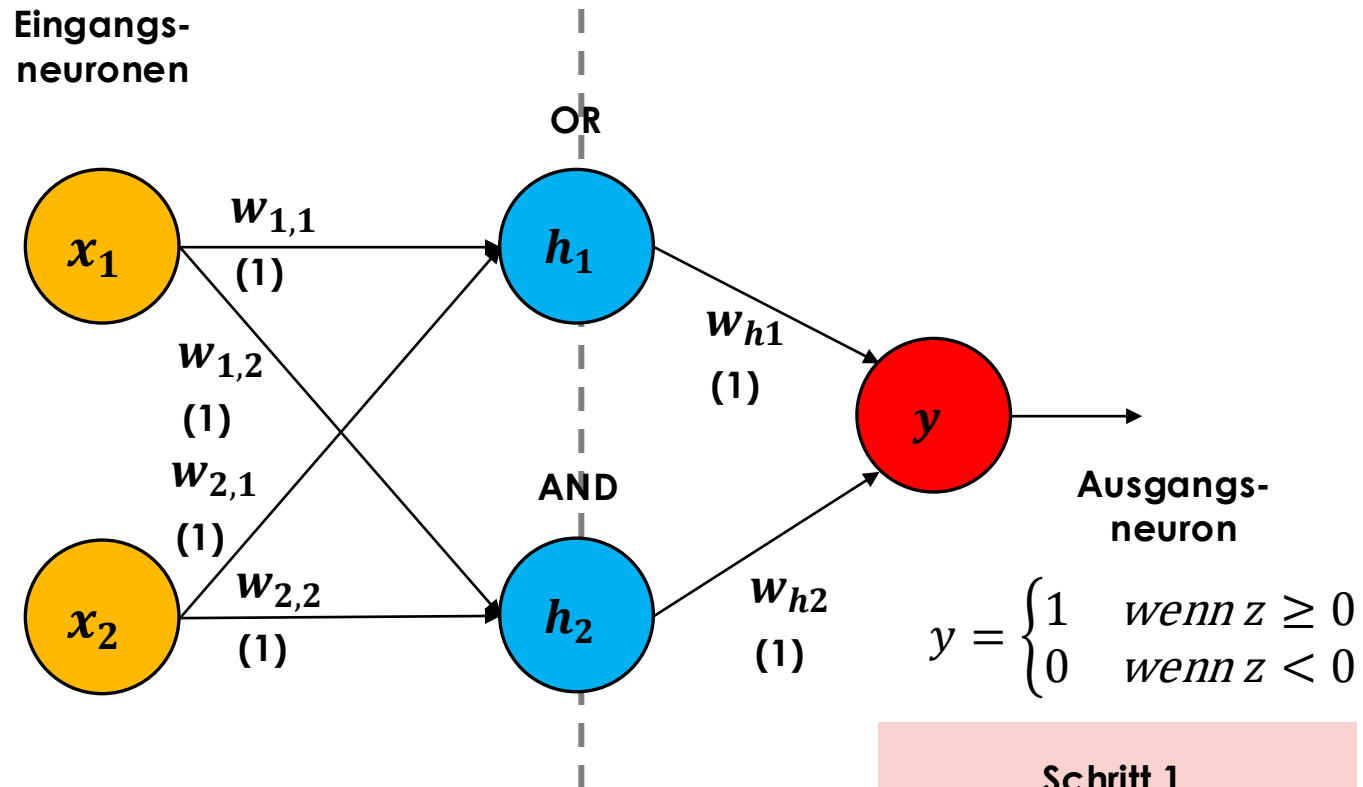
2) OR ($b=-0.5$)



3) XOR



Aktivierungsfunktion



$$y = \begin{cases} 1 & \text{wenn } z \geq 0 \\ 0 & \text{wenn } z < 0 \end{cases}$$

Schritt 1

$$z = w_1 x_1 + w_2 x_2 + b$$

Schritt 2

$$f(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

◆ Was passiert ohne Aktivierungsfunktion?

Der Output ist einfach: $y = z$

Wenn wir mehrere Schichten hintereinander haben:

👉 Also immer noch **eine lineare Funktion**.

◆ Was macht die Aktivierungsfunktion?

- entscheidet, ob ein Neuron „aktiv“ wird
- führt eine **Nichtlinearität** ein
- ermöglicht komplexe Entscheidungsgrenzen

Beispiel:

Step-Funktion:

$$f(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

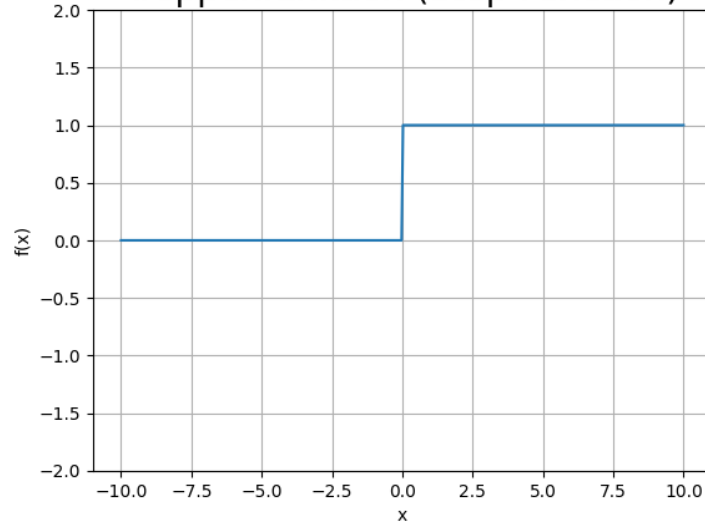
Oder moderner:

ReLU: $f(z) = \max(0, z)$

Diese Funktionen sind **nicht linear**.

Aktivierungsfunktion

Treppenfunktion (Step Function)



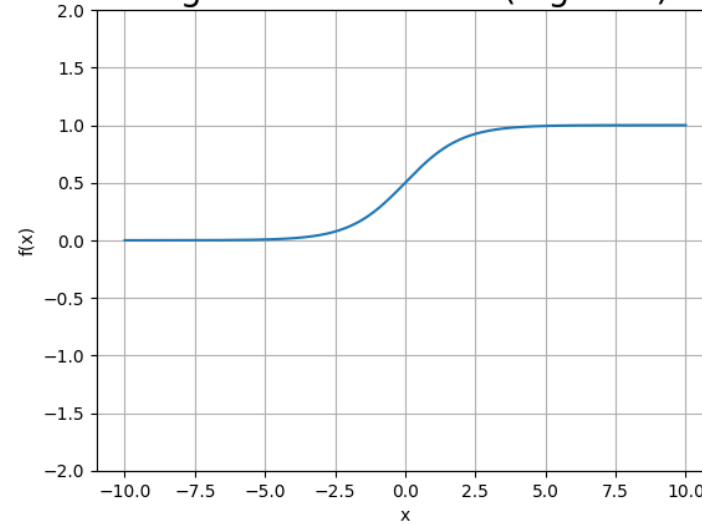
$$f(x) = \begin{cases} 0 & \text{für } x < 0 \\ 1 & \text{für } x \geq 0 \end{cases}$$

Die Schrittfunktion gibt nur zwei mögliche Werte aus:

- 0 für negative Eingaben
- 1 für positive oder null Eingaben

DEEP LEARNING MIT PYTHON UND TENSORFLOW

Logistische Funktion (Sigmoid)

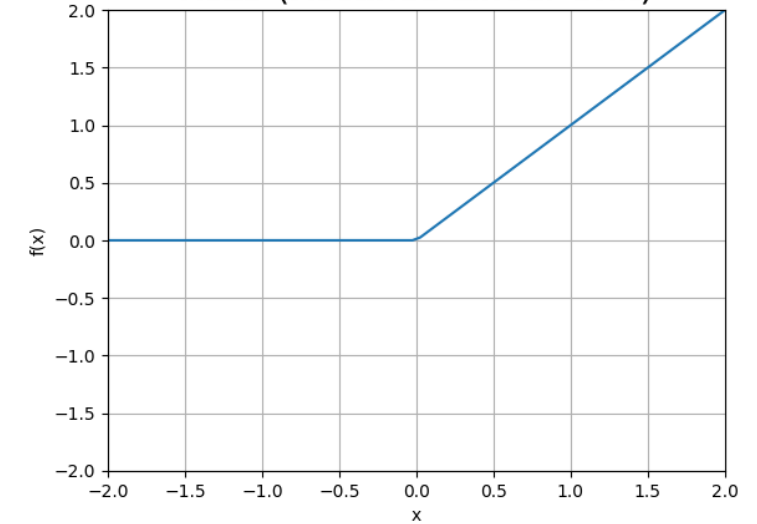


$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Die Funktion transformiert jede reelle Zahl in einen Wert zwischen 0 und 1.

- Sehr negative Werte → nahe 0
- Sehr positive Werte → nahe 1
- Bei $x = 0 \rightarrow 0,5$

ReLU (Rectified Linear Unit)



$$f(x) = \begin{cases} 0 & \text{für } x < 0 \\ x & \text{für } x \geq 0 \end{cases}$$

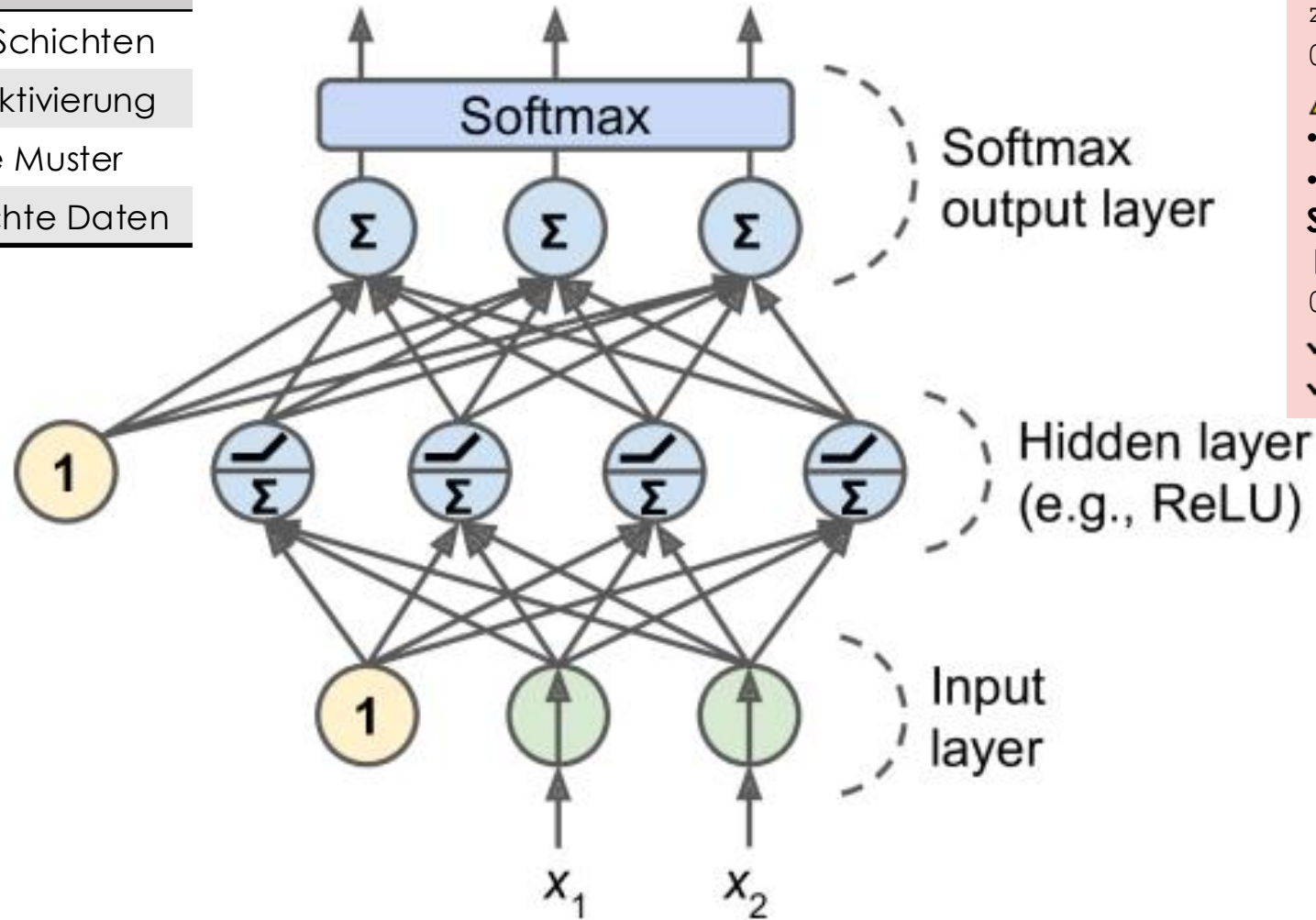
ReLU schneidet alle negativen Werte ab.

- Negative Eingaben → 0
- Positive Eingaben → unverändert
- Halb linear, halb abgeschnitten.

(MLP) Perzeptron

MLP

mehrere Schichten
weiche Aktivierung
komplexe Muster
gut für echte Daten



2 Softmax – Aktivierungsfunktion in der Output Layer

Schritt 3: Output-Scores (Logits)

Für jede Klasse einen Score:

$z_{\text{out}} = [2.1, -1.3, 0.2, 3.8, -0.5, -2.0, 1.1, 0.3, 0.9, 2.5]$

⚠ Das sind **keine Wahrscheinlichkeiten**:

- können negativ sein
- Summe ist beliebig

Schritt 4: Softmax anwenden

$[0.08, 0.01, 0.03, 0.42, 0.02, 0.01, 0.07, 0.03, 0.06, 0.27]$

✓ Alle Werte zwischen 0 und 1

✓ **Summe = 1**

1 ReLU – Aktivierungsfunktion in der Hidden Layer

Schritt 1: Lineare Berechnung

$z = [-2.3, 0.7, 4.1, -0.5]$

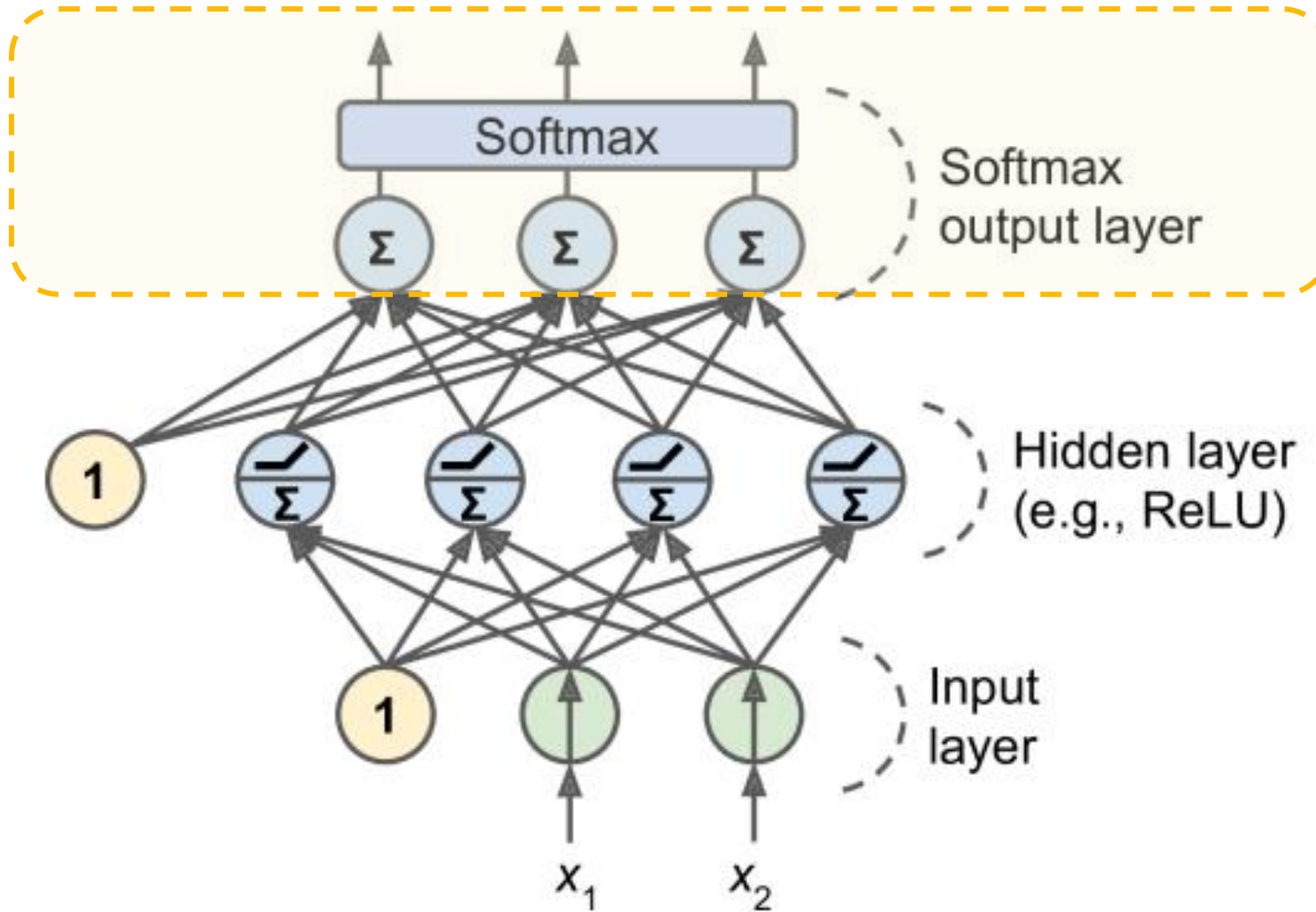
Schritt 2: ReLU anwenden

$h = [0, 0.7, 4.1, 0]$

🔍 Was macht ReLU?

- **Negativen Werte: 0**
- Positive Werte bleiben erhalten
- **Keine Wahrscheinlichkeiten**
- **Summe ist egal**

(MLP) Perzeptron



Softmax

Was passiert, wenn wir **mehr als zwei Klassen** haben?

Sigmoid: zwei Klassen

Softmax: drei Klassen oder mehr

Die letzte Schicht gibt 10 Zahlen aus:

[2.1, 0.3, 5.4, -1.2, ...]

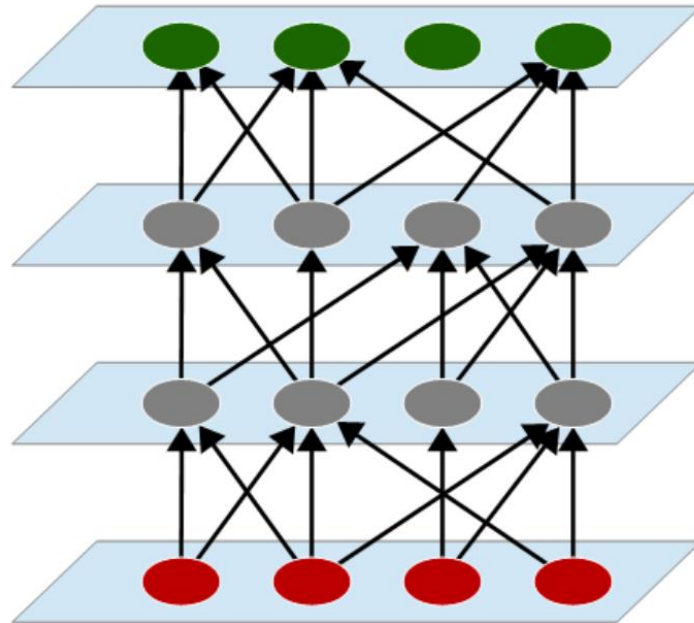
Softmax macht **Wahrscheinlichkeiten** daraus:

[0.01, 0.002, 0.85, 0.00001, ...]

- Nur positive Werte
- Alle Werte zwischen 0 und 1
- Die Summe ist genau 1

Deep Learning

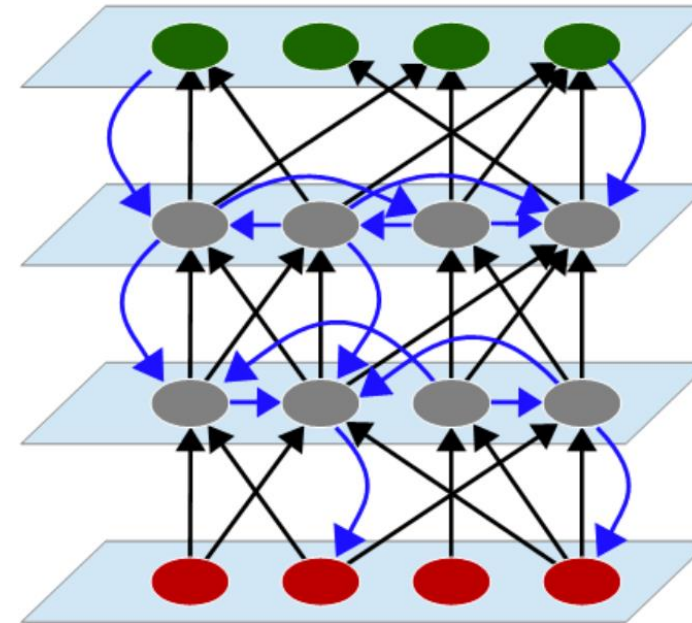
Deep Learning



Vorwärtsgerichtete Architektur

(Quelle: Helmut Linde)

Gehirn



Rekurrente Architektur

Grundlagen TensorFlow

Get started with TensorFlow

TensorFlow makes it easy to create ML models that can run in any environment. Learn how to use the intuitive APIs through interactive code samples.

View tutorials >

Webseite: <https://www.tensorflow.org/>

```
import tensorflow as tf
mnist = tf.keras.datasets.mnist

(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0

model = tf.keras.models.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(x_train, y_train, epochs=5)
model.evaluate(x_test, y_test)
```

Run quickstart

12:30–13:10 ☕ Mittagspause

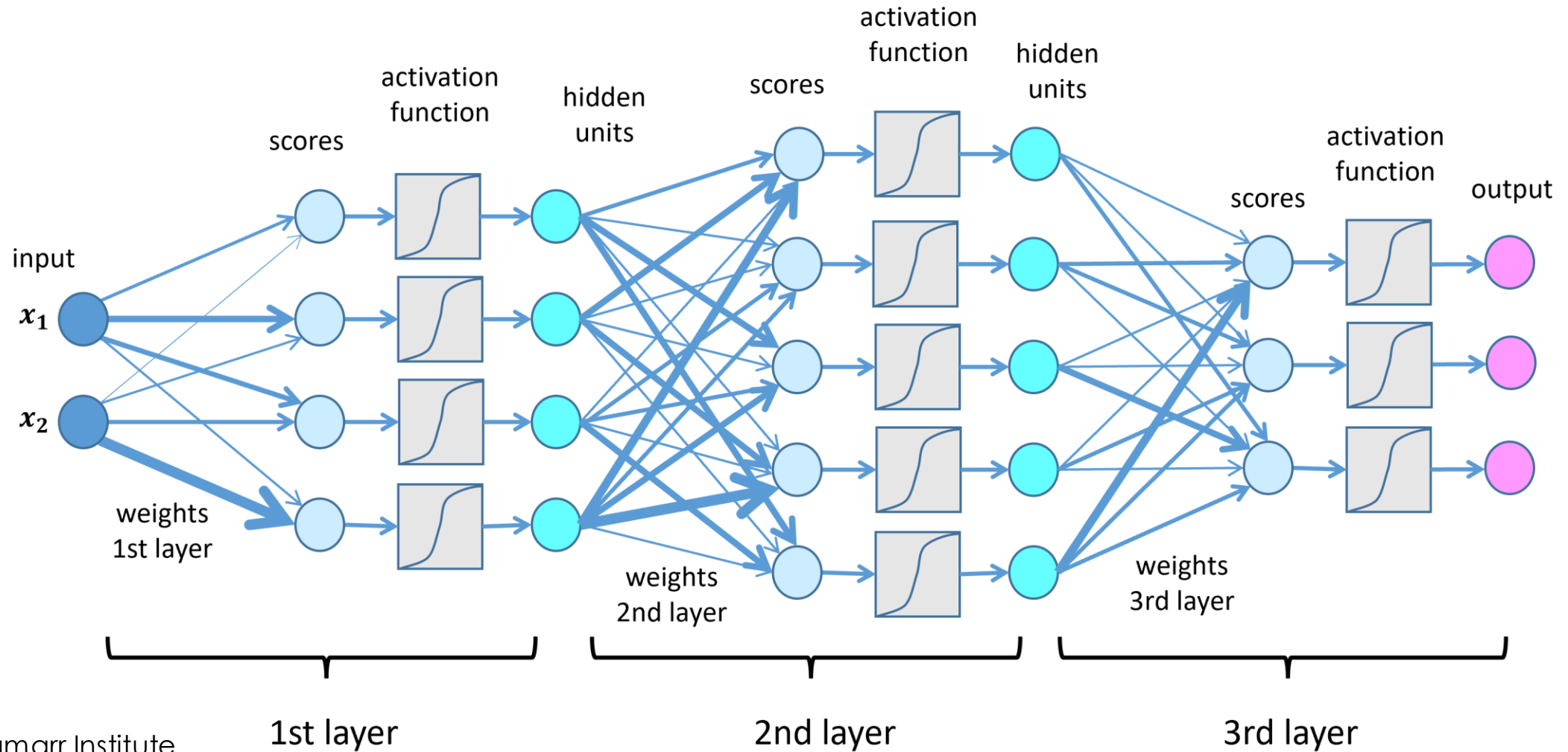




Tag 2 13:10–14:15

Einführung in Deep Learning

Deep Learning – Vom Datensatz zum Modell



Label

Daten

0

T_shirt



1

Trouser



2

Pullover



3

Dress



4

Coat



5

Sandal



6

Shirt



7

Sneaker



8

Bag



9

Ankle boot

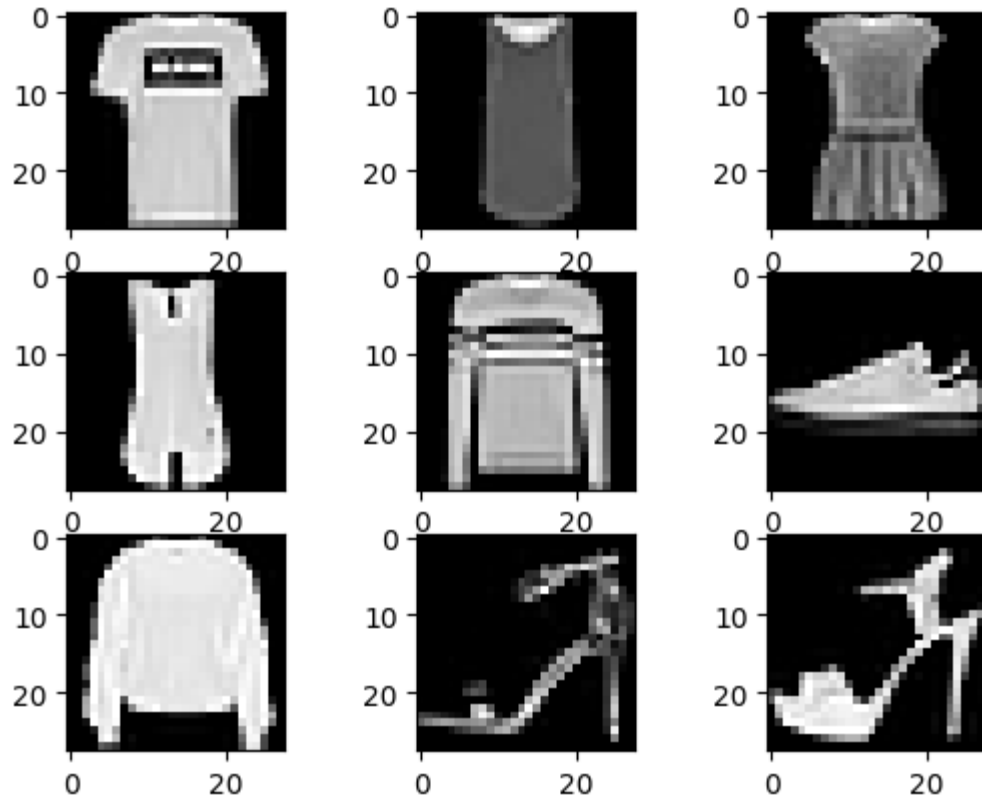


Trainingdaten

Testdaten

Daten

Fashion-MNIST ist ein Datensatz mit Bildern von Kleidungsstücken, der häufig verwendet wird, um Deep Learning und neuronale Netze zu erklären und zu üben.



Welche Daten enthält Fashion-MNIST?

Fashion-MNIST besteht aus vielen kleinen Bildern, und jedes Bild zeigt ein einzelnes Kleidungsstück.

Umfang des Datensatzes

60.000 Trainingsbilder (zum Lernen) + 10.000 Testbilder (zum Testen)

Die Bilder

Größe: 28 × 28 Pixe

Graustufen: Jeder Pixel hat einen Wert von 0 bis 255

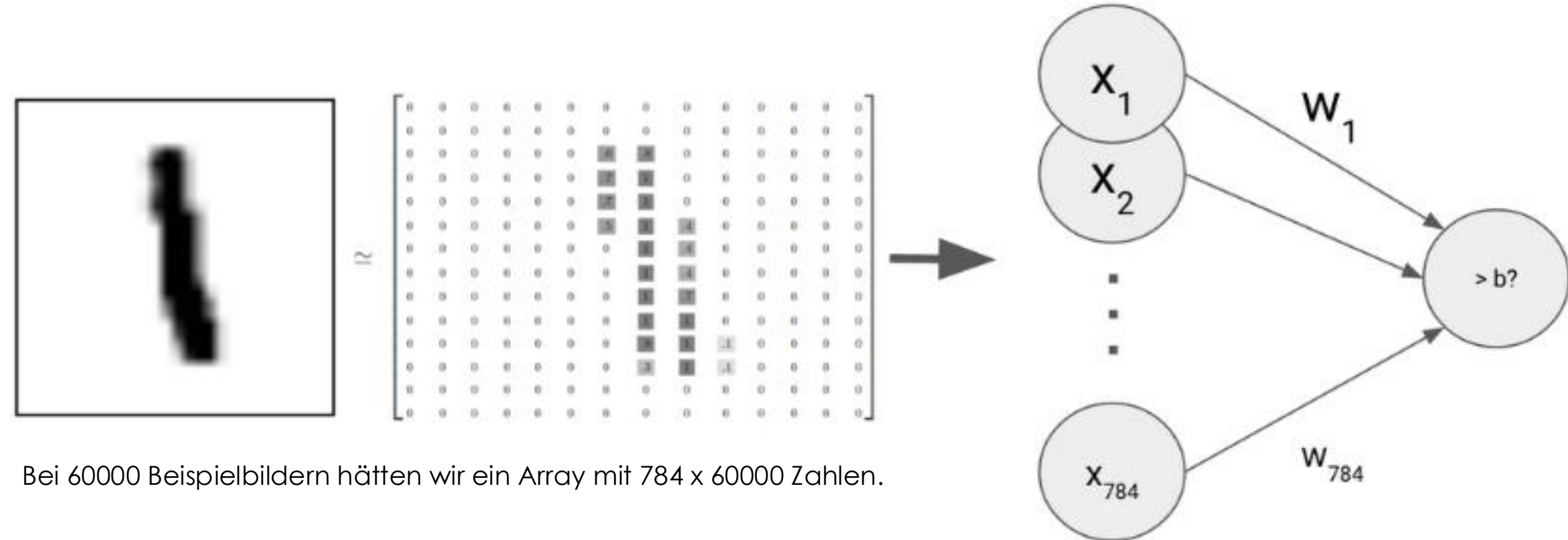
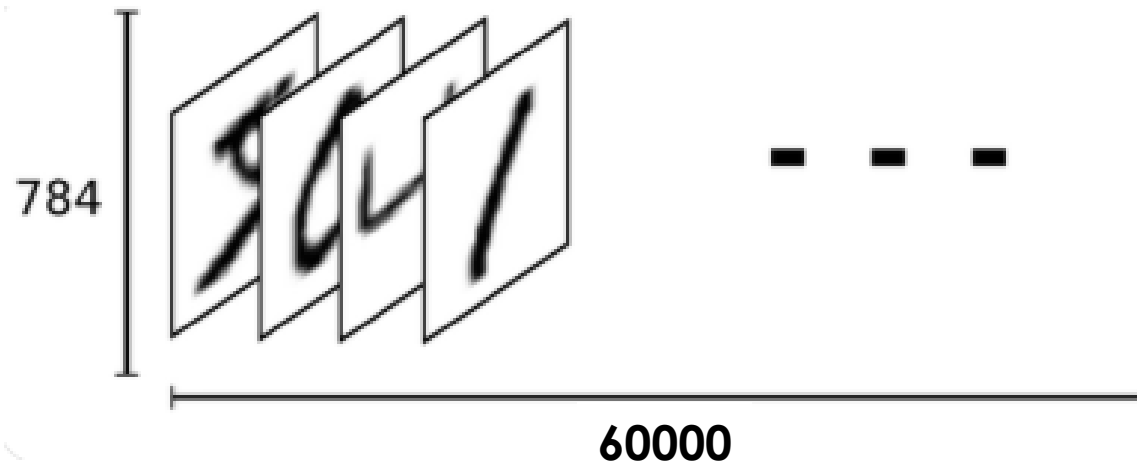
0 = schwarz ; 255 = weiß

 Das 28×28-Bild wird in eine lange Zahlenliste mit 784 Werten umgewandelt.

 Die Klassen (Labels)

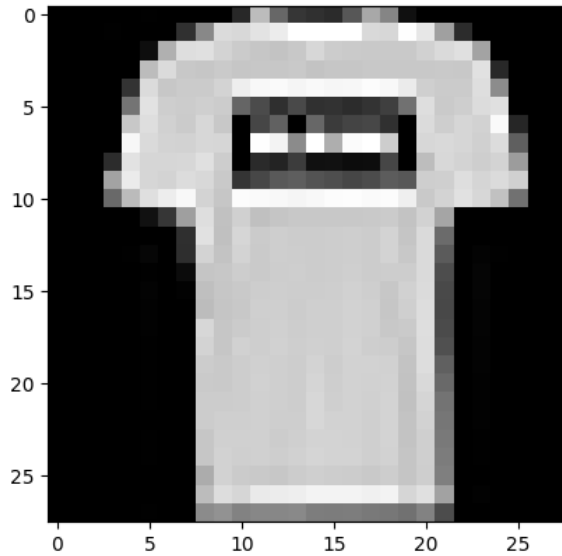
Das neuronale Netz soll:

 lernen, das richtige Kleidungsstück auf einem Bild zu erkennen

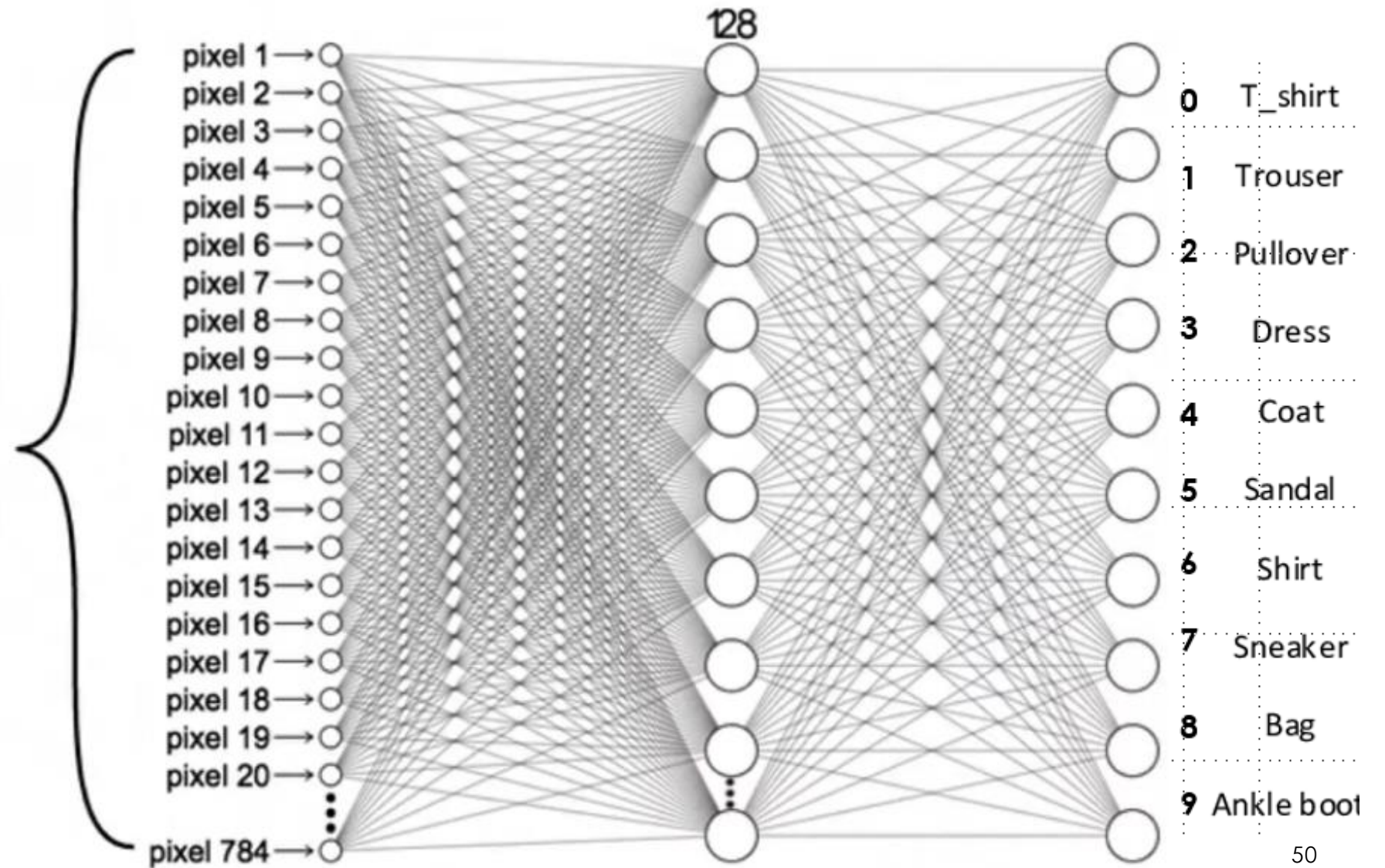


Bei 60000 Beispielbildern hätten wir ein Array mit 784 x 60000 Zahlen.

Daten



DEEP LEARNING MIT PYTHON UND TENS



Ergebnis: Prognosen



Lernmethoden und wichtige Modelle (Überblick)

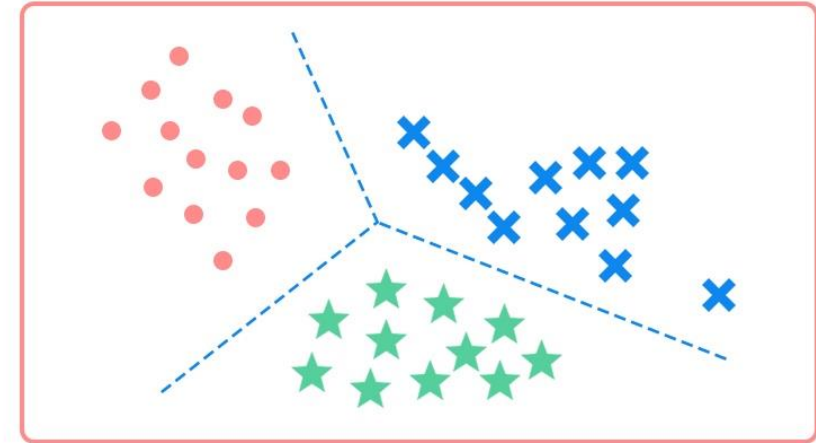
1. Supervised Learning (Überwachtes Lernen)

- **Trainingsdaten haben Labels**
- Modell lernt Eingabe → Zielwert
- Fehler wird berechnet (Loss Function)

Beispiele:

- Bildklassifikation
- Spam-Erkennung

Classification



Supervised learning

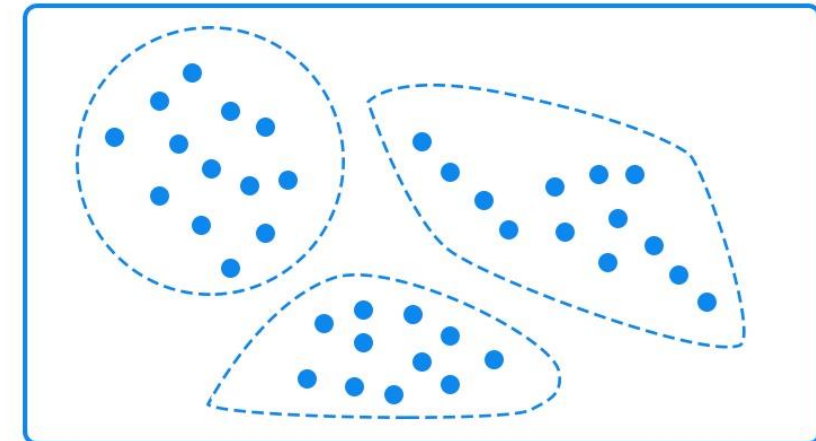
2. Unsupervised Learning (Unüberwachtes Lernen)

- **Keine Labels**
- Modell sucht selbst Strukturen
- Muster- oder Clustererkennung

Beispiele:

- Kundensegmentierung

Clustering



Unsupervised learning

(Bildquelle)

Wichtige Modelle - FNN

Feedforward Neural Network (FNN)

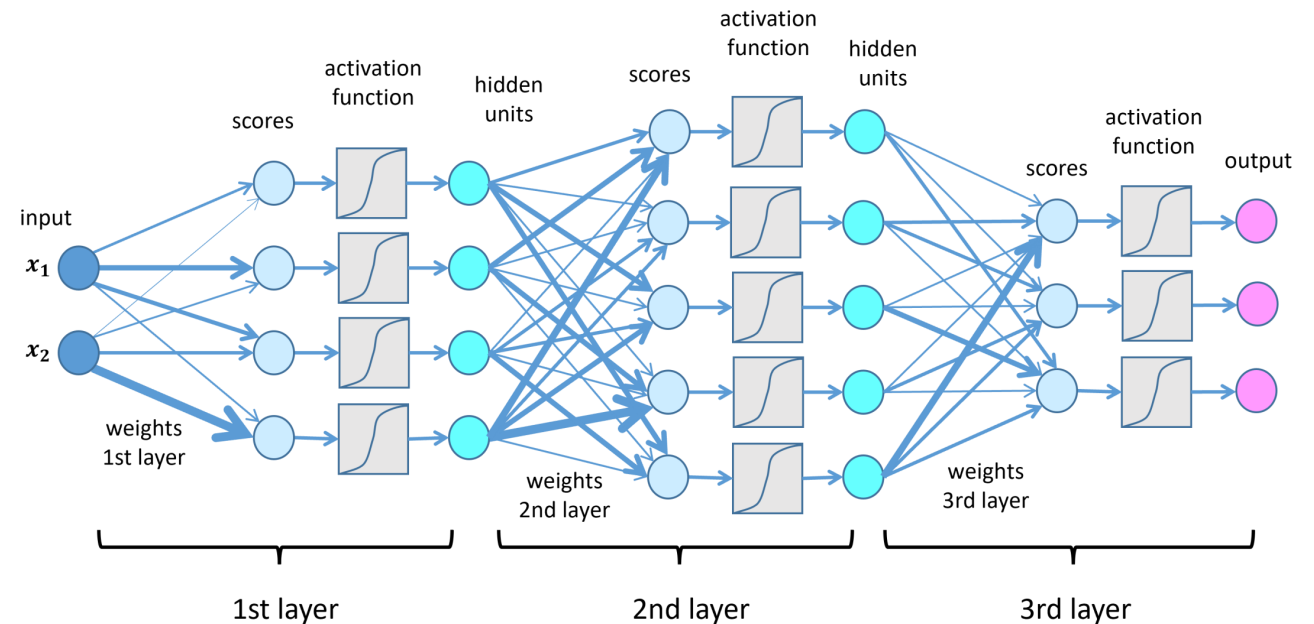
- Daten fließen von links nach rechts – vom **Input** über versteckte Schichten zum **Output**.

Was passiert?

- Jede Schicht verarbeitet Informationen
- Gewichte werden angepasst
- Am Ende entsteht eine Vorhersage

Typische Anwendungen

- Klassifikation (z. B. Spam oder kein Spam)
- Z.B. Preisvorhersage



© Lamarr Institute

Wichtige Modelle - CNN

Convolutional Neural Network (CNN)

- Das Modell erkennt **Muster in Bildern**.

Was macht es besonders?

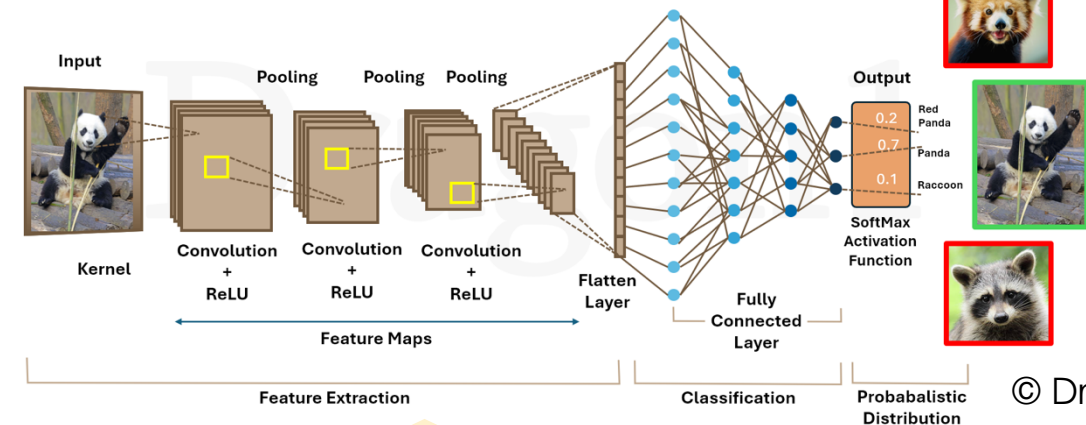
- Arbeitet mit Filtern
- Erkennt Kanten, Formen, Texturen
- Baut schrittweise komplexe Strukturen auf

Typische Anwendungen

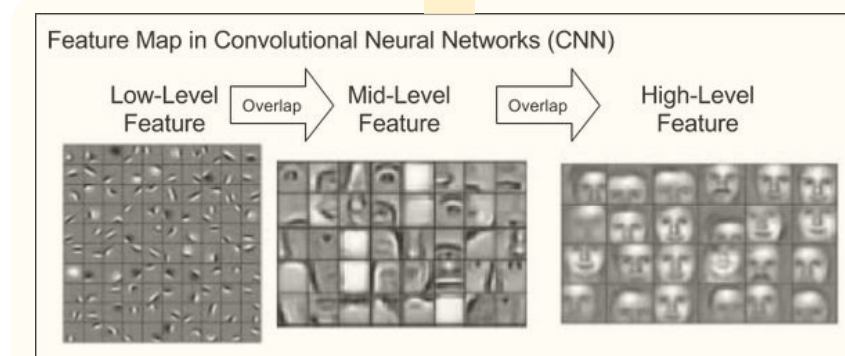
- Gesichtserkennung
- Medizinische Bildanalyse
- Autonomes Fahren

Convolutional Neural Networks

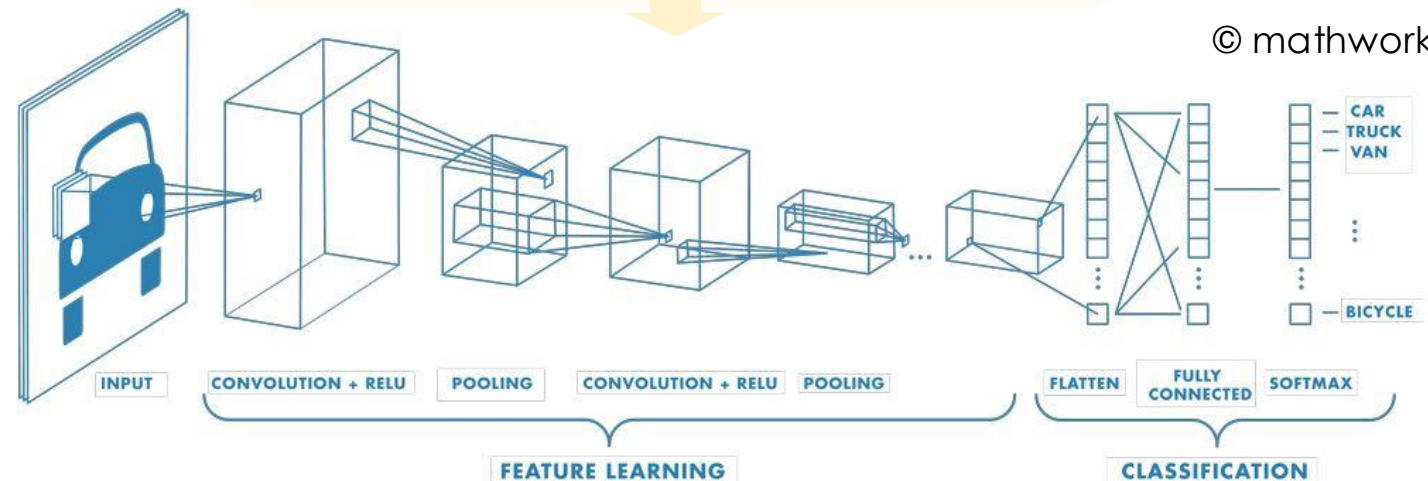
(AI Deep Learning)



© Dragon1



© mathworks



Wichtige Modelle - RNN

Recurrent Neural Network (RNN)

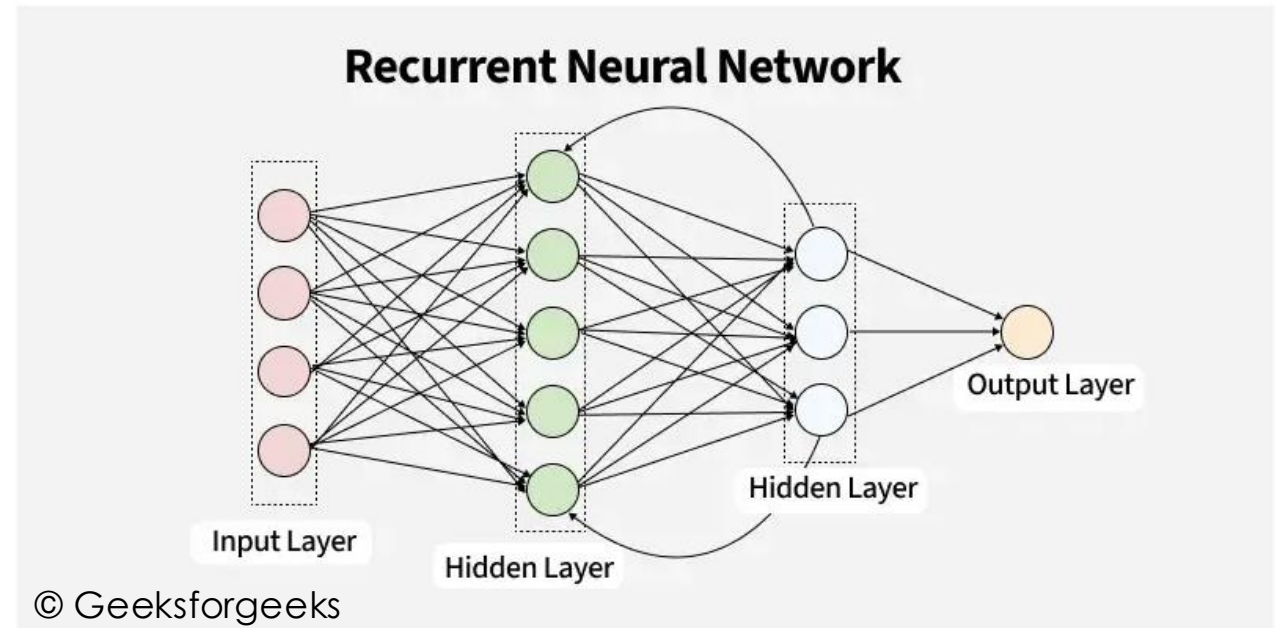
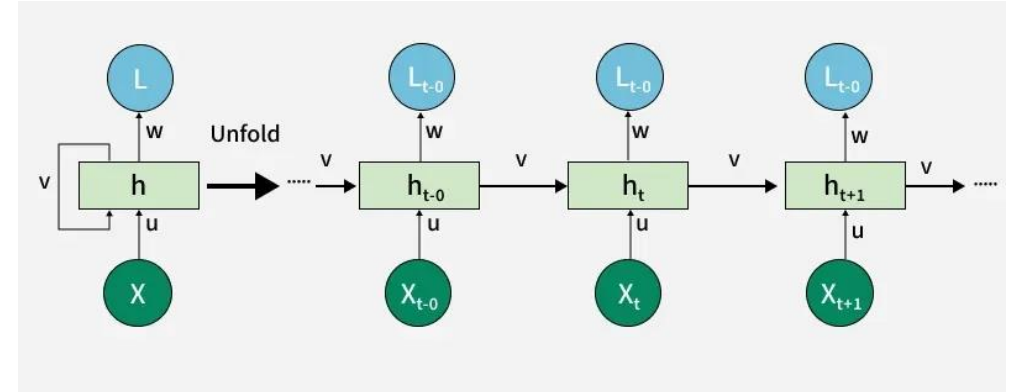
- Das Modell verarbeitet **Informationen in Reihenfolge**.

Was ist besonders?

- Es „merkt sich“ vorherige Eingaben
- Frühere Informationen beeinflussen spätere Entscheidungen

Typische Anwendungen

- Sprache
- Zeitreihen (z. B. Aktienkurse)
- Textanalyse



Wichtige Modelle (Überblick)

Transformer – Moderne Sprachmodelle

- Alle Wörter werden gleichzeitig betrachtet.

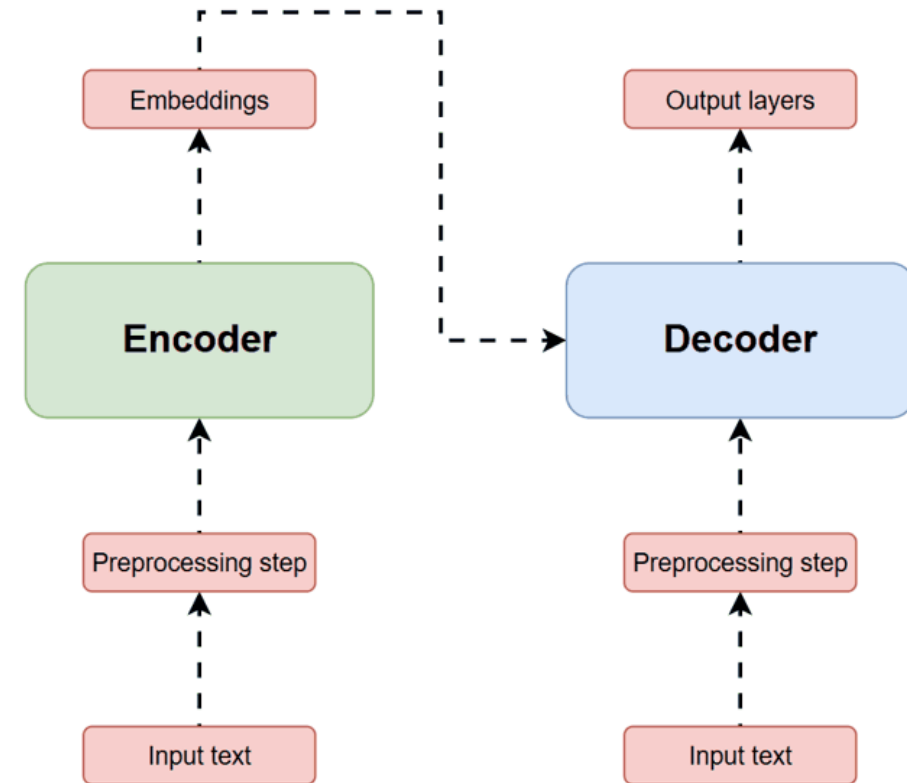
★ Was macht ihn so stark?

- Attention-Mechanismus
- Erkennt Zusammenhänge über große Distanzen
- Sehr effizient bei großen Datenmengen

🎯 Typische Anwendungen

- Chatbots
- Übersetzung
- Textzusammenfassung
- Large Language Models

© compute.hivenet



Lernen aus Daten: Wie trainiert ein Modell?

Wie wird aus Daten ein „intelligentes“ Modell?

Trainingsdaten vs. Testdaten

Trainingsdaten

- Werden zum Lernen verwendet
- Modell passt seine Gewichte an
- Kennt die richtigen Antworten

Testdaten

- Werden nicht zum Lernen benutzt
- Prüfen die Generalisierungsfähigkeit
- Simulieren neue, unbekannte Daten

Die Grundidee des Lernens

Lernprinzip

1. Modell macht eine Vorhersage
2. Vorhersage wird mit der Wahrheit verglichen
3. Modell bekommt Feedback
4. Modell verbessert sich

Datenqualität ist wichtiger als Modellgröße

Ein großes Modell hilft nicht, wenn:

- Daten falsch sind
- Daten unvollständig sind
- Klassen unausgeglichen sind

Modellbewertung – Wie gut ist ein Modell?

Wie wissen wir, ob ein Modell gut ist?

Ein Modell ist nicht gut, nur weil es trainiert wurde.

Wir müssen es bewerten.

Ein Modell wird:

- Mit Trainingsdaten gelernt
- Mit Testdaten bewertet

Überanpassung (Overfitting)

Ein Modell kann:

- Auf Trainingsdaten sehr gut sein
- Aber auf Testdaten schlecht

Das nennt man Overfitting.

Wichtige Bewertungsmetriken (Confusion Metriken für Klassifikation)

- **Accuracy:** Anteil korrekt vorhergesagter Beispiele.

Stell dir vor: Wir erkennen Spam-Mails.

- **Precision:** Wie viele der als Spam erkannten Mails sind wirklich Spam?
- **Recall:** Wie viele der tatsächlichen Spam-Mails wurden erkannt?
- **F1-Score:** Kombination aus Precision und Recall.

Modellgröße ≠ Qualität

-  Ein größeres Modell ist nicht automatisch besser.

Entscheidend sind:

- Datenqualität
- Ausgewogene Klassen
- Saubere Labels



Epoch
000,000

Learning rate
0.003

Activation
Sigmoid

Regularization
None

Regularization rate
0

Problem type
Classification

Epoch:

Wie oft das Modell die komplette Trainingsmenge schon durchlaufen hat.

Regularization: Ein Mechanismus gegen Overfitting - hier ausgeschaltet = 0. (Genau das Overfitting-Thema aus unserem Modellbewertungs-Slide!)

Learning rate:

Wie groß die Schritte sind, mit denen sich das Modell bei jeder Korrektur bewegt. So eine kleine Zahl heißt: kleine, vorsichtige Schritte — dauert länger, ist aber stabiler.

Activation:

Erinnert euch an unser Perzeptron-Beispiel (Alarm ja/nein)?

Sigmoid: "weiche" Version davon statt hart 0 oder 1 zu entscheiden, gibt sie jeden Wert dazwischen aus (0,5 = "sich nicht sicher").

ReLU: negative Werte einfach auf 0 abschneidet.

DA
Which
you want to use?



Ratio of training to
test data: 80%

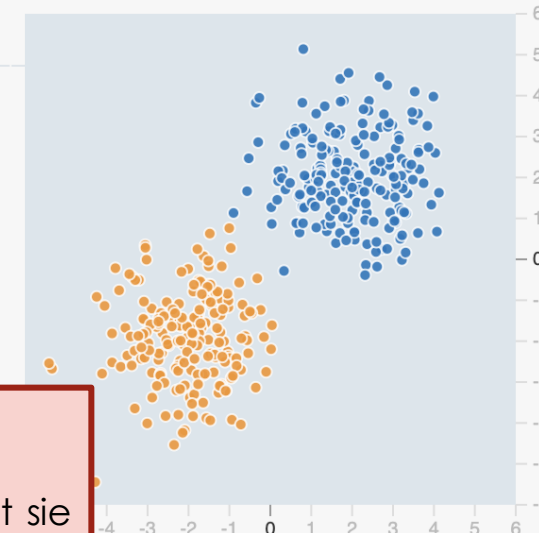
Noise: 10

Batch size: 9

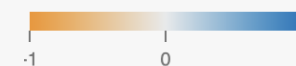
REGENERATE

1 neuron

Training loss 0.509



Colors shows
data, neuron and
weight values.



☐ Show test data

☐ Discretize output



Epoch
000,000

Learning rate
0.003

Activation
Sigmoid

Regularization
None

Regularization rate
0

Problem type
Classification

DATA

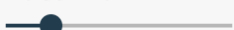
Which dataset do you want to use?



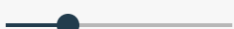
Ratio of training to test data: 80%



Noise: 10



Batch size: 9



REGENERATE

FEATURES

Which properties do you want to feed in?

Ratio of training to test data (80%) 80 % der Punkte werden zum Lernen benutzt, 20 % bleiben zum ehrlichen Testen zurück - genau das Prinzip Trainingsdaten vs. Testdaten.

Noise (10) Wie viel "Unordnung" in die Daten gemischt wird - simuliert echte, nicht perfekte Daten.

Batch size (9) Wie viele Datenpunkte das Modell sich ansieht, bevor es seine Gewichte einmal aktualisiert.

+ - 1 HIDDEN LAYER

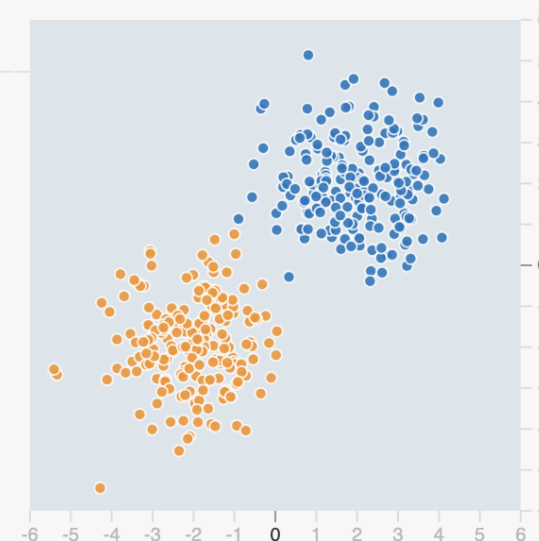


1 neuron

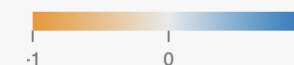
OUTPUT

Test loss 0.528

Training loss 0.509



Colors shows data, neuron and weight values.



☐ Show test data

☐ Discretize output



Epoch
000,000

Learning rate

0.003

Activation

Sigmoid

Regularization

None

Regularization rate

0

Problem type

Classification

DATA

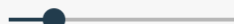
Which dataset do you want to use?



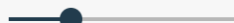
Ratio of training to test data: 80%



Noise: 10



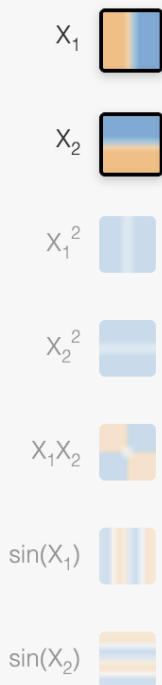
Batch size: 9



REGENERATE

FEATURES

Which properties do you want to feed in?



1 HIDDEN LAYER



1 neuron



This is the output from one neuron. Hover to see it larger.

OUTPUT

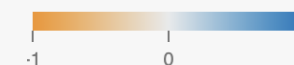
Test loss 0.528
Training loss 0.509

OUTPUT

Test loss 0,528 / Training loss 0,509
Wie viele Fehler das Modell macht.
Die Grafik zeigt die "Entscheidungsgrenze" - wo das Modell orange von blau trennt.



Colors shows data, neuron and weight values.



☐ Show test data

☐ Discretize output



Epoch
000,000

Learning rate
0.003

Activation
Sigmoid

Regularization
None

Regularization rate
0

Problem type
Classification

DATA

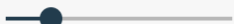
Which dataset do you want to use?



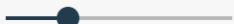
Ratio of training to test data: 80%



Noise: 10



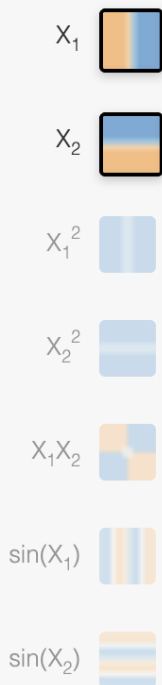
Batch size: 9



REGENERATE

FEATURES

Which properties do you want to feed in?



+ - 1 HIDDEN LAYER

+ -

1 neuron

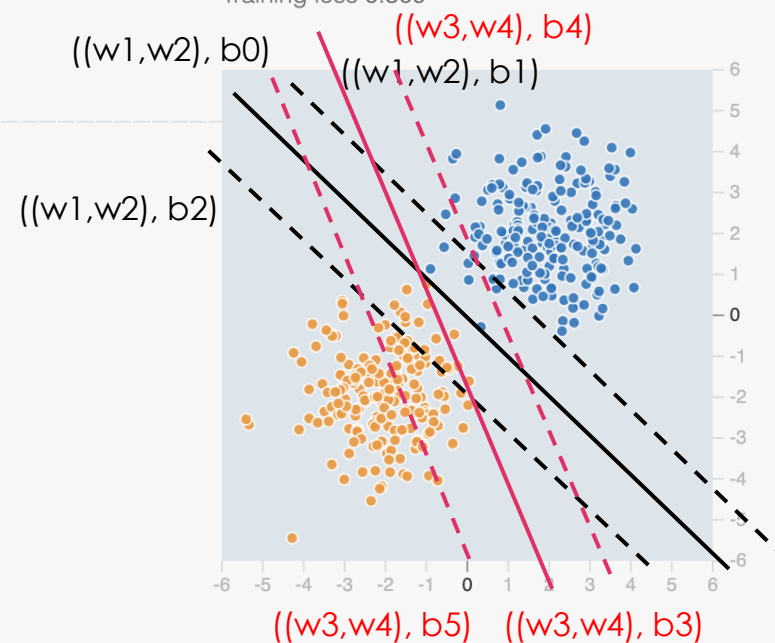
This is the output from one neuron. Hover to see it larger.

$$w_1x_1 + w_2x_2 > b$$

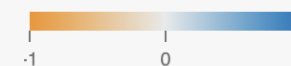
OUTPUT

Test loss 0.528

Training loss 0.509



Colors shows data, neuron and weight values.



☐ Show test data

☐ Discretize output



Epoch
000,000

Learning rate
0.03

Activation
ReLU

Regularization
None

Regularization rate
0

Problem type
Classification

DATA

Which dataset do you want to use?



Ratio of training to test data: 50%



Noise: 0



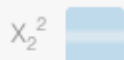
Batch size: 10



REGENERATE

FEATURES

Which properties do you want to feed in?



+ - 1 HIDDEN LAYER

+ -

3 neurons

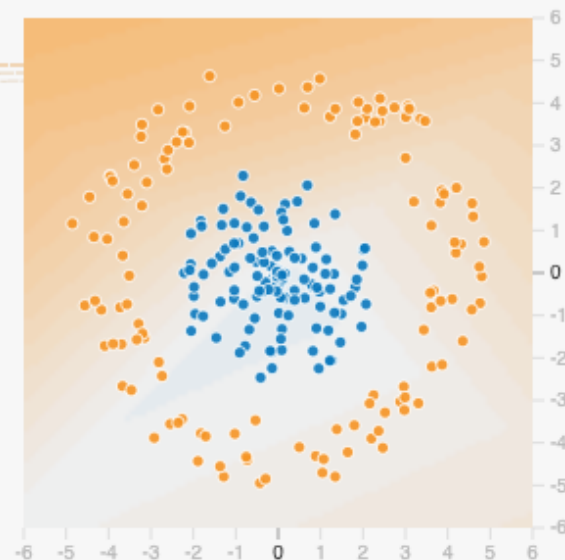


This is the output from one neuron.
Hover to see it larger.

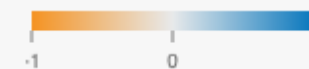
OUTPUT

Test loss 0.432

Training loss 0.413

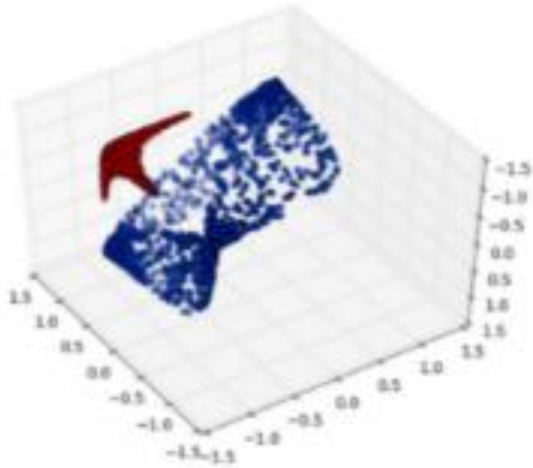
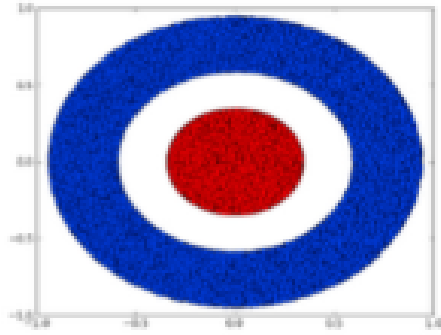


Colors shows data, neuron and weight values.

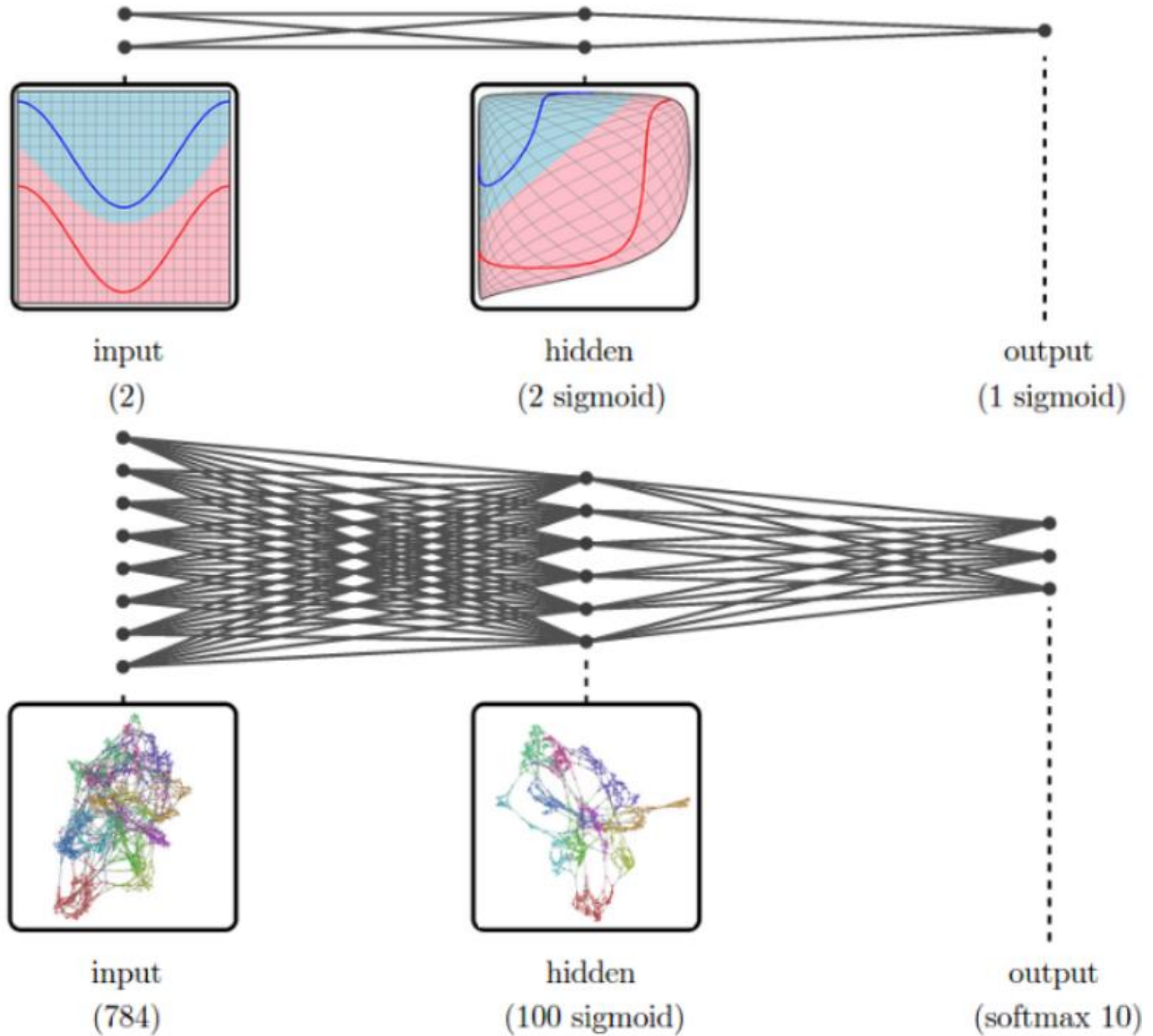


☐ Show test data

☐ Discretize output



DEEP LEARNING MIT PYTHON UND TENSORFLOW



15:00–15:20 🍴 Pause





Tag 2 15:20–16:40

Praktische Übung: Einfaches Modell

- Modell bauen & trainieren mit TensorFlow
- Modellbewertung & Interpretation
- Ergebnisse analysieren
- Verbesserungen diskutieren

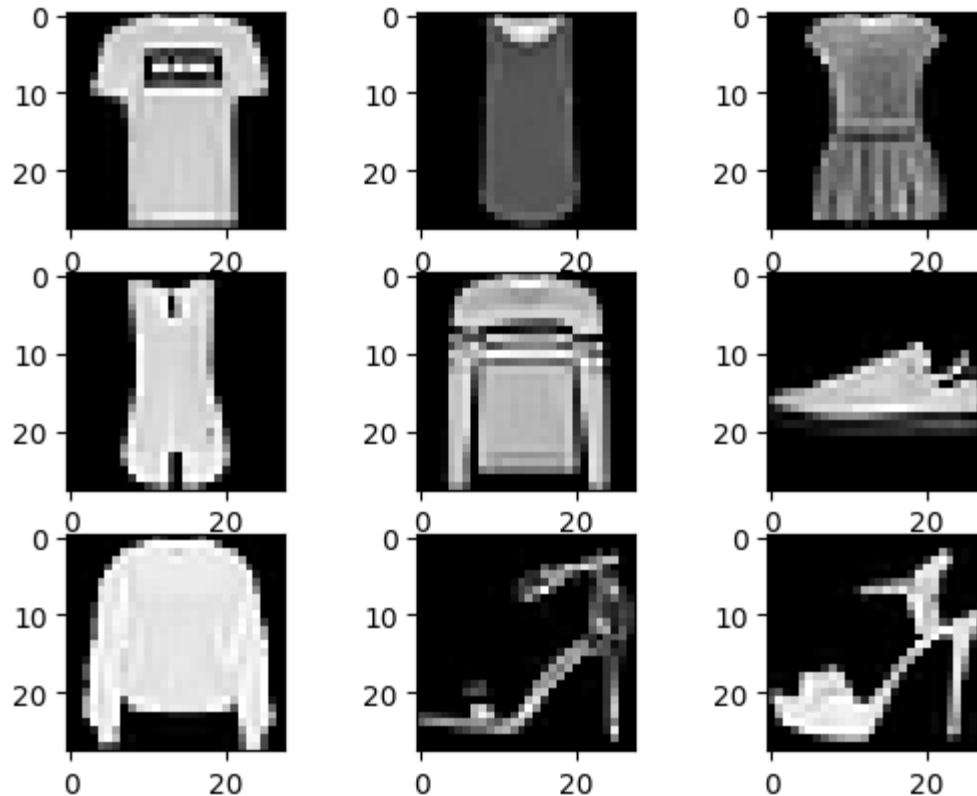
Daten: Fashion MNIST

Label

0	T_shirt										
1	Trouser										
2	Pullover										
3	Dress										
4	Coat										
5	Sandal										
6	Shirt										
7	Sneaker										
8	Bag										
9	Ankle boot										

Daten: Fashion MNIST

Fashion-MNIST ist ein Datensatz mit Bildern von Kleidungsstücken, der häufig verwendet wird, um Deep Learning und neuronale Netze zu erklären und zu üben.



Welche Daten enthält Fashion-MNIST?

Fashion-MNIST besteht aus vielen kleinen Bildern, und jedes Bild zeigt ein einzelnes Kleidungsstück.

Umfang des Datensatzes

60.000 Trainingsbilder (zum Lernen) + 10.000 Testbilder (zum Testen)

Die Bilder

Größe: 28 × 28 Pixe

Graustufen: Jeder Pixel hat einen Wert von 0 bis 255

0 = schwarz ; 255 = weiß

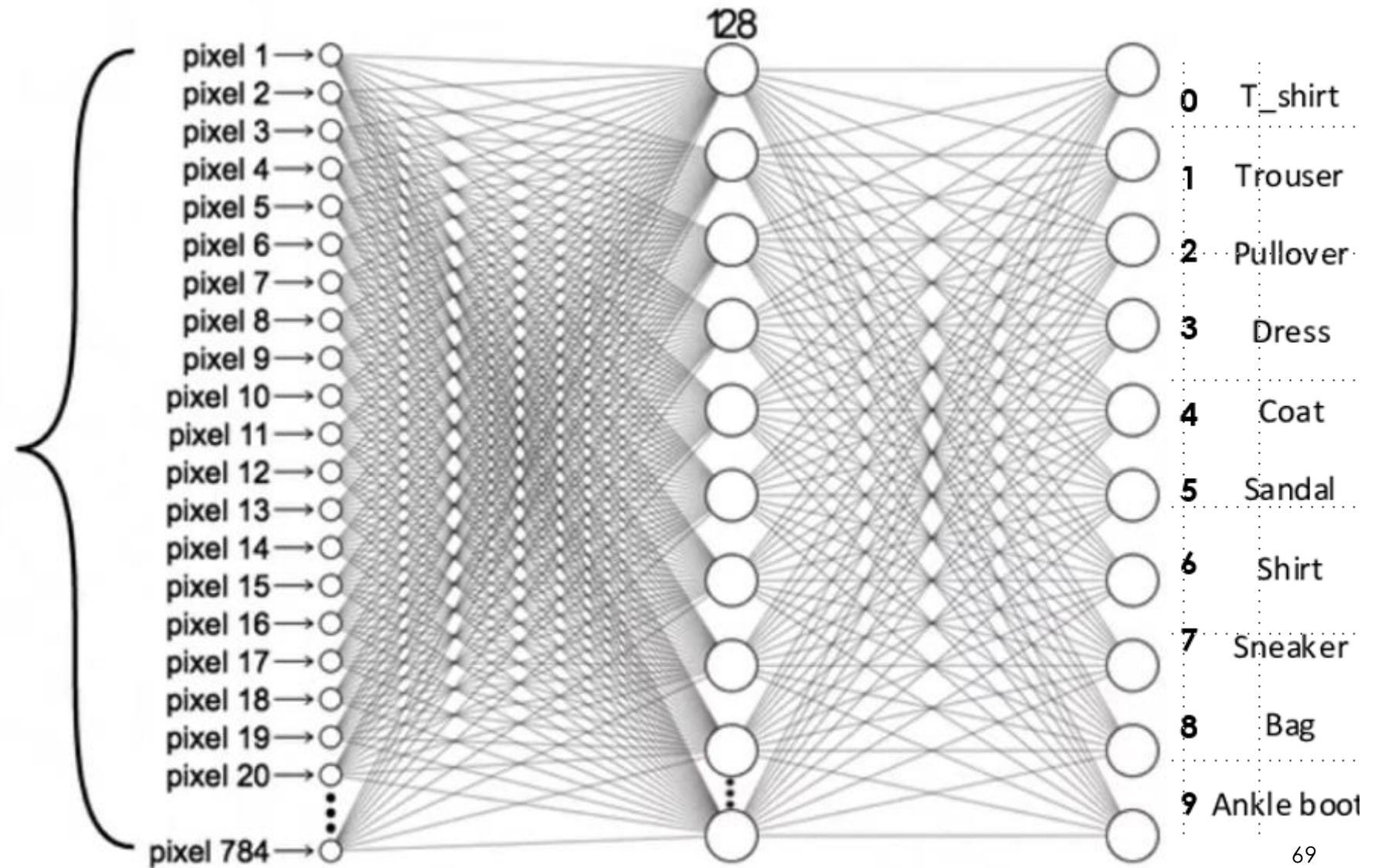
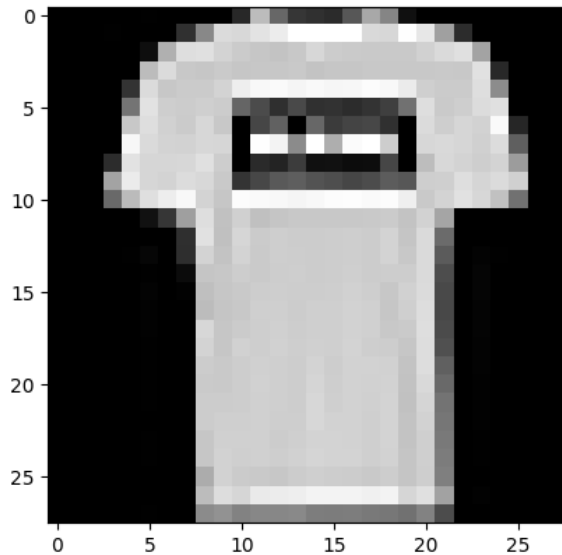
👉 Das 28×28-Bild wird in eine lange Zahlenliste mit 784 Werten umgewandelt.

📌 Die Klassen (Labels)

Das neuronale Netz soll:

👉 lernen, das richtige Kleidungsstück auf einem Bild zu erkennen

Daten: Fashion MNIST





Tag 2 16:40–17:00

Ergebnisse & Diskussion

- Präsentation
- Reflexion
- Q&A